Universidad de los Andes · Maestría en Ingeniería de Software (MISO)

MISO4304 – DevOps: Agilizando el despliegue continuo de aplicaciones

Proyecto 1 · Entrega 1
Despliegue manual de un microservicio sobre AWS Elastic Beanstalk (PaaS)

Microservicio de lista negra global de emails

Grupo 16	
Integrante 1	[Nombre completo]
Integrante 2	[Nombre completo]
Integrante 3	[Nombre completo]
Integrante 4	[Nombre completo]
Fecha	Octubre de 2026

Información general de la entrega

Repositorio GitHub	`https://github.com/dcabarcasb/MISO4304_Grupo16_team_devops`
URL de la aplicación (Beanstalk)	`http://blacklist-grupo16.us-east-1.elasticbeanstalk.com`
Documentación publicada en Postman	`https://documenter.getpostman.com/view/58993096/2sBYHSx1BC`
Colección y environments versionados	`postman/collection.json`, `postman/environment.json` (local), `postman/environment.aws.json` (AWS)
Región AWS	us-east-1 (N. Virginia)

Arquitectura desplegada

- **Aplicación:** microservicio Python 3.12 con Flask, Flask-RESTful, Flask-SQLAlchemy, Flask-Marshmallow, Flask-JWT-Extended y Werkzeug, servido con gunicorn.
- **Endpoints:** `POST /blacklists` (agrega un email; guarda IP de origen y fecha/hora), `GET /blacklists/<email>` (indica si está bloqueado y el motivo) y `GET /health` (health check sin token; devuelve versión e instancia). Los endpoints de negocio exigen Bearer Token (JWT firmado a partir de un token estático).
- **PaaS:** AWS Elastic Beanstalk, ambiente `blacklist-env` (aplicación `blacklist-app`), plataforma *Python 3.12 running on 64bit Amazon Linux 2023*, tipo *Load balanced* con Application Load Balancer y Auto Scaling de **3 a 6** instancias `t3.micro`.
- **Base de datos:** Amazon RDS PostgreSQL 18.3 (`blacklist-db`, `db.t4g.micro`, Single-AZ, sin acceso público), accesible solo desde el security group de las instancias de Beanstalk.

1. Creación y despliegue de la aplicación en AWS Elastic Beanstalk

1.a Configuración de RDS

Se creó la base de datos desde *RDS → Create database* con el método *Full configuration*, motor PostgreSQL y plantilla *Free tier*.

Parámetro	Valor
Identificador	`blacklist-db`
Motor / versión	PostgreSQL 18.3
Clase / almacenamiento	`db.t4g.micro`, 20 GiB gp2, sin autoscaling de almacenamiento
Disponibilidad	Single-AZ (us-east-1a)
Usuario maestro	`postgres` (contraseña auto-administrada, no versionada)
Base de datos inicial	`blacklist`
Red	VPC por defecto, **sin acceso público**, security group nuevo `blacklist-rds-sg`, puerto 5432
Endpoint	`blacklist-db.caj6aqockm6d.us-east-1.rds.amazonaws.com:5432`

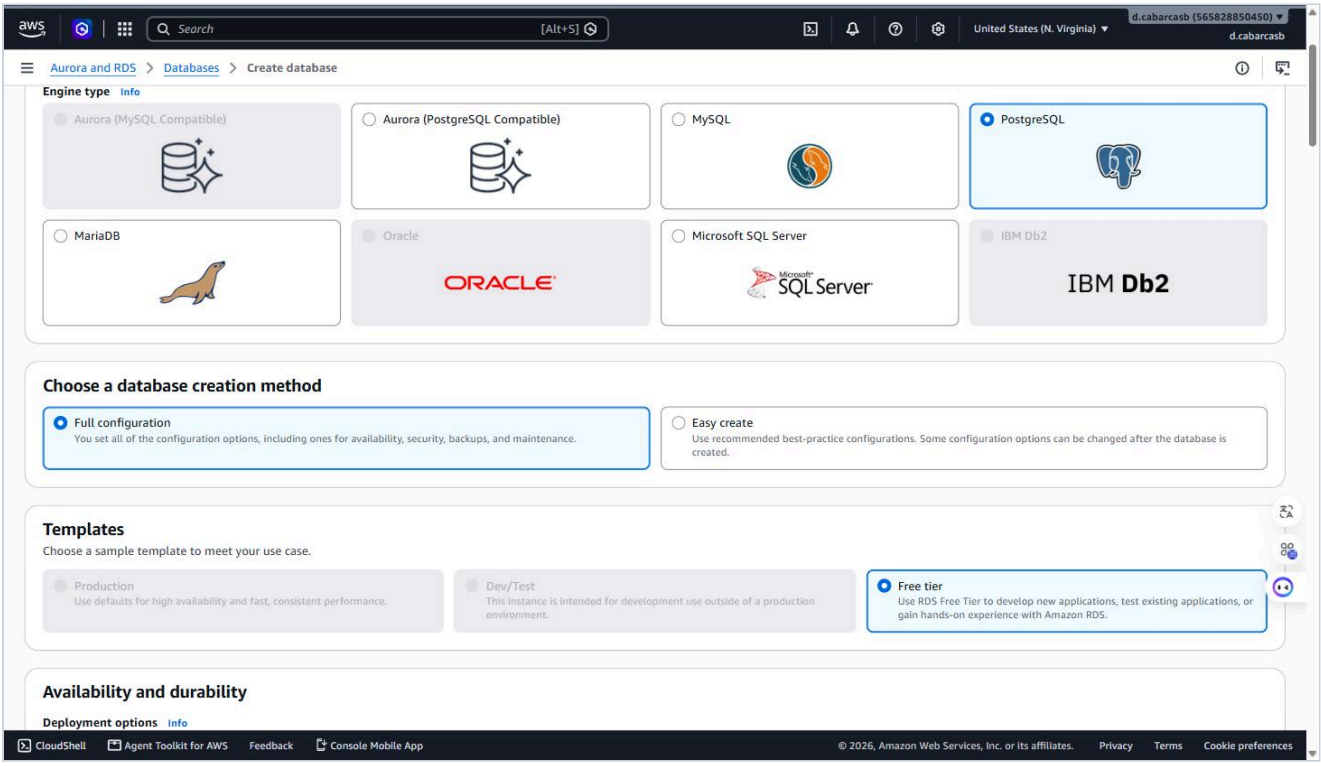


Figura 1. Motor PostgreSQL, método Full configuration y plantilla Free tier.

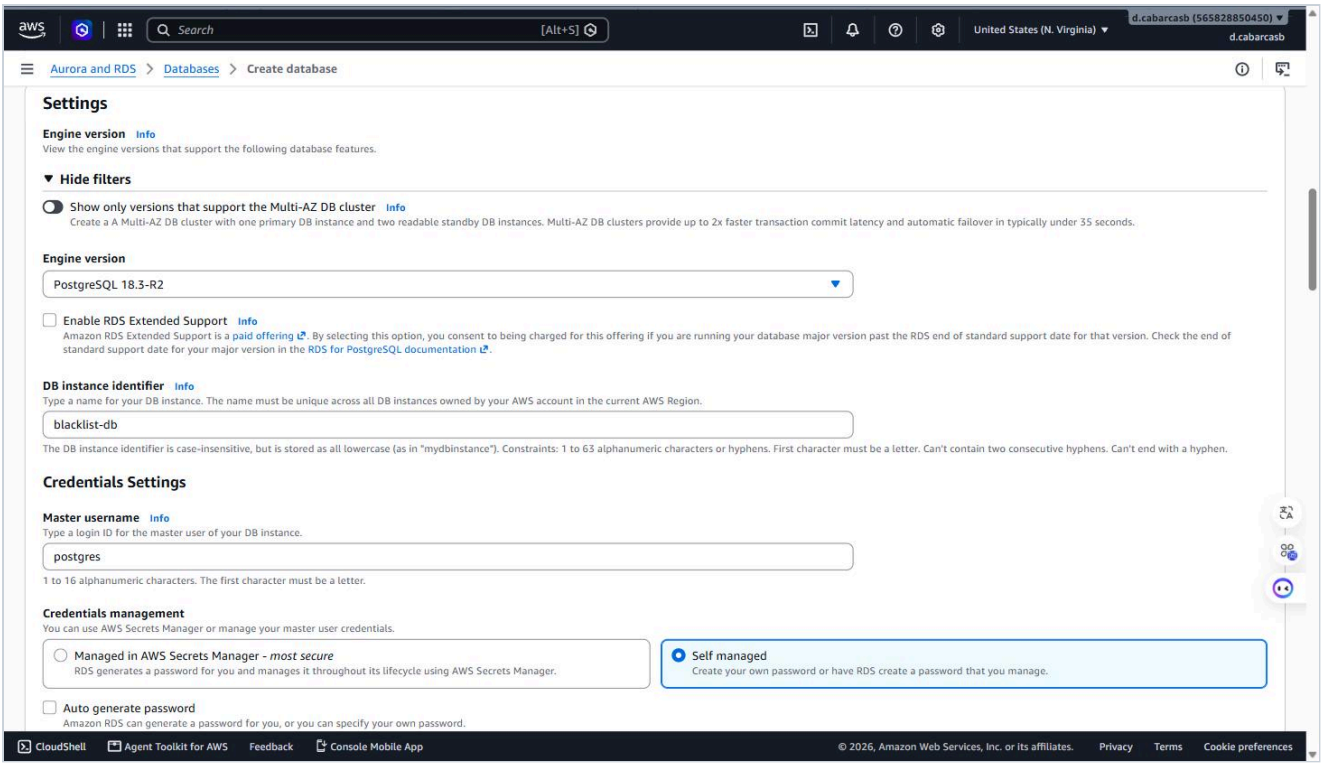


Figura 2. Versión del motor, identificador de la instancia y usuario maestro.

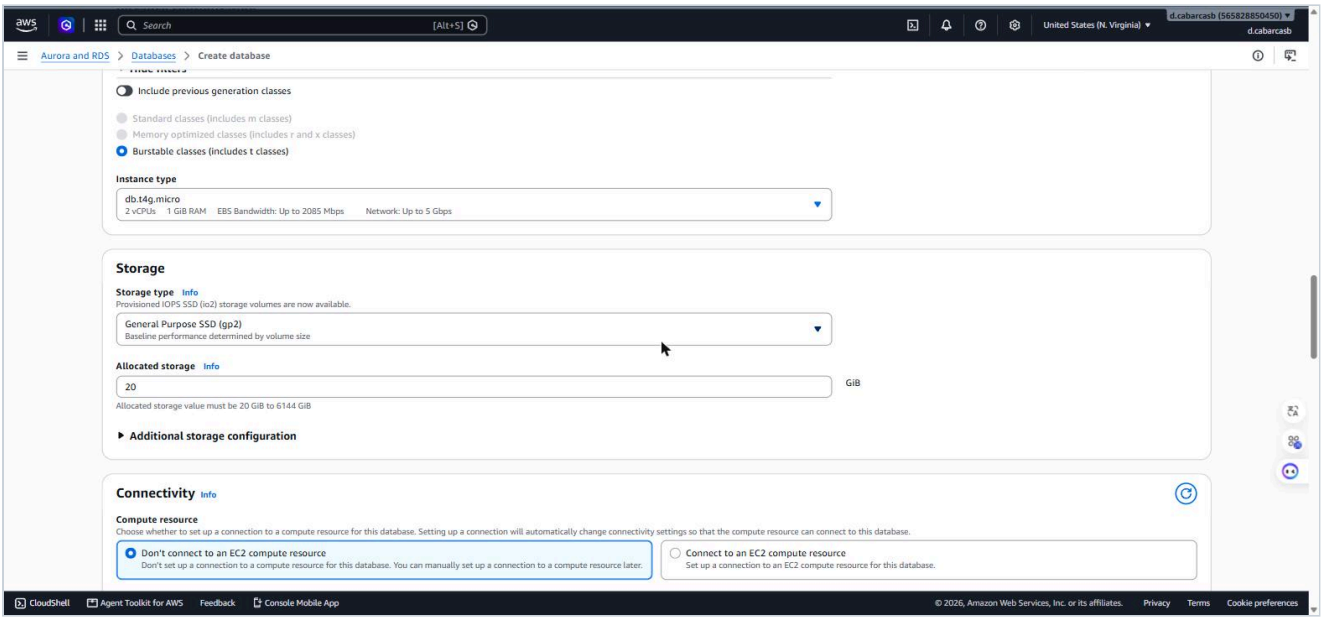


Figura 3. Clase de instancia db.t4g.micro y almacenamiento de 20 GiB.

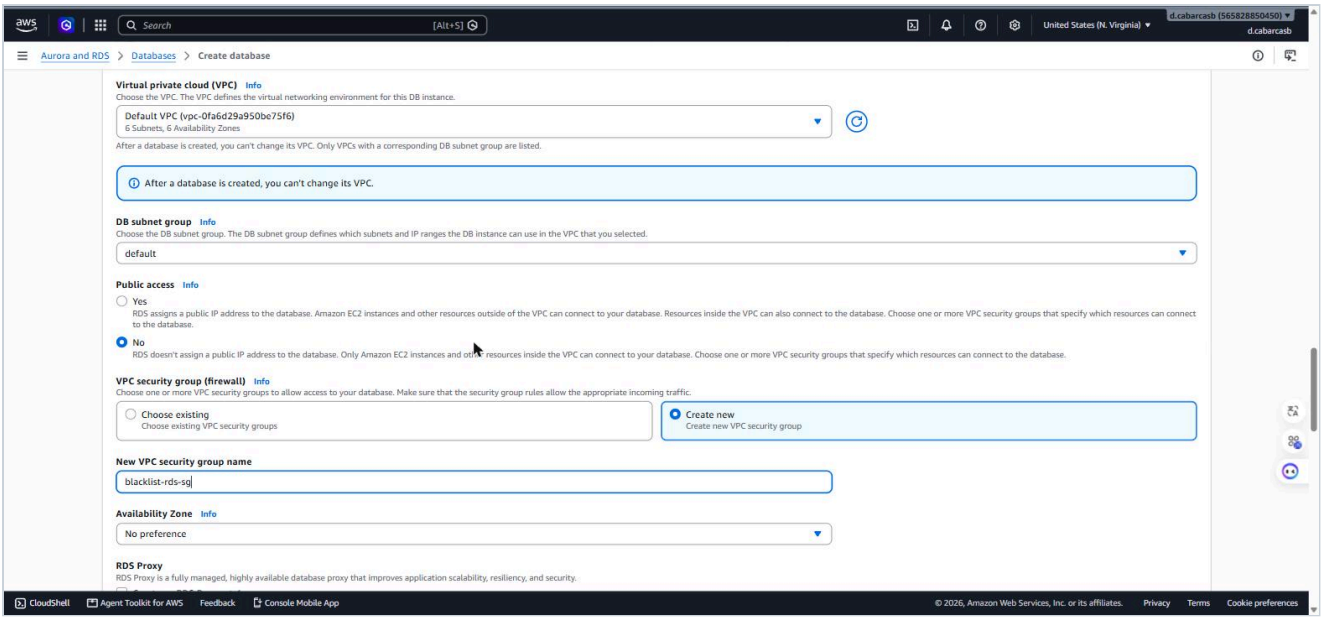


Figura 4. Conectividad: VPC por defecto, sin acceso público y security group blacklist-rds-sg.

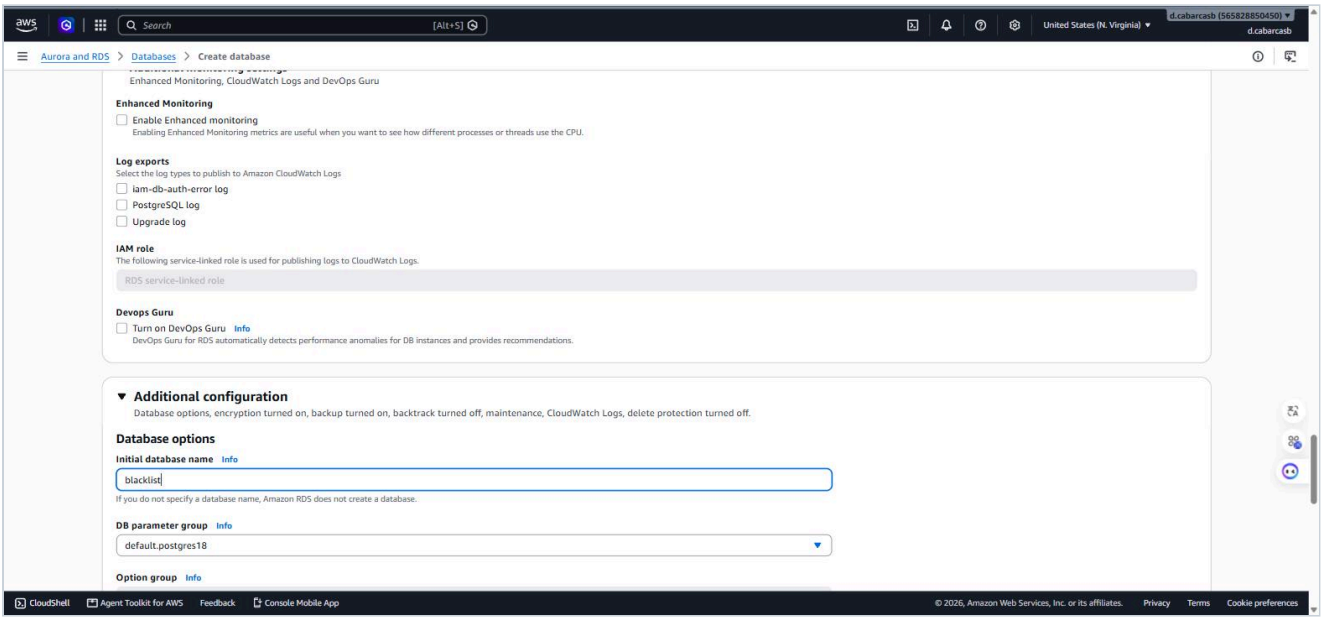


Figura 5. Configuración adicional: base de datos inicial `blacklist`.

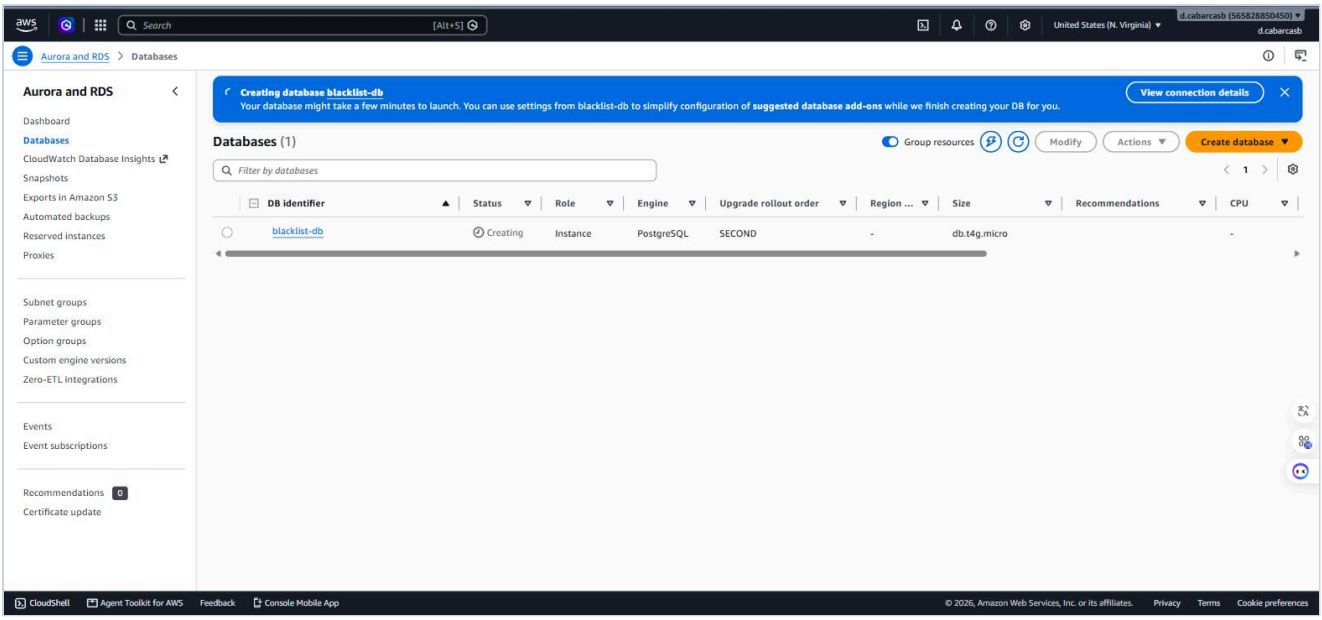


Figura 6. Base de datos en estado Creating.

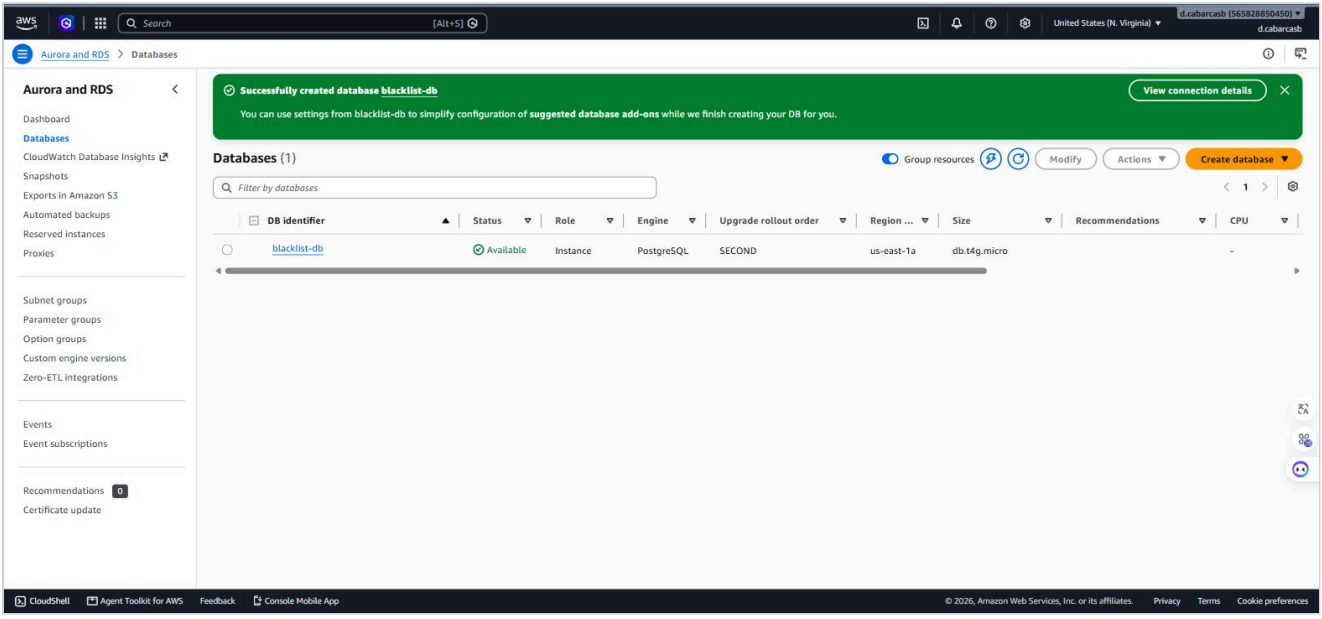


Figura 7. Base de datos disponible (Available).

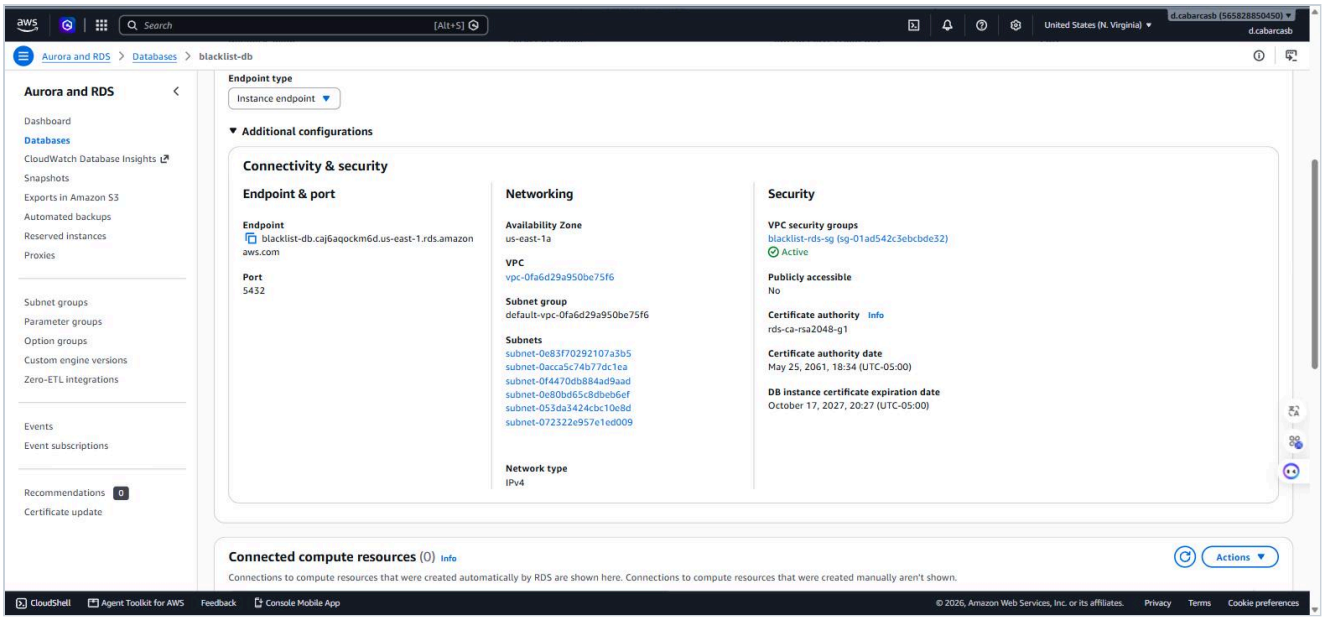


Figura 8. Endpoint, puerto y security group de la instancia RDS.

Una vez creado el ambiente de Beanstalk se agregó al security group de RDS una regla de entrada **TCP 5432 cuyo origen es el security group de las instancias de Beanstalk** (`sg-0f28005a04a27d7c3`), de modo que solo la aplicación puede conectarse a la base de datos.

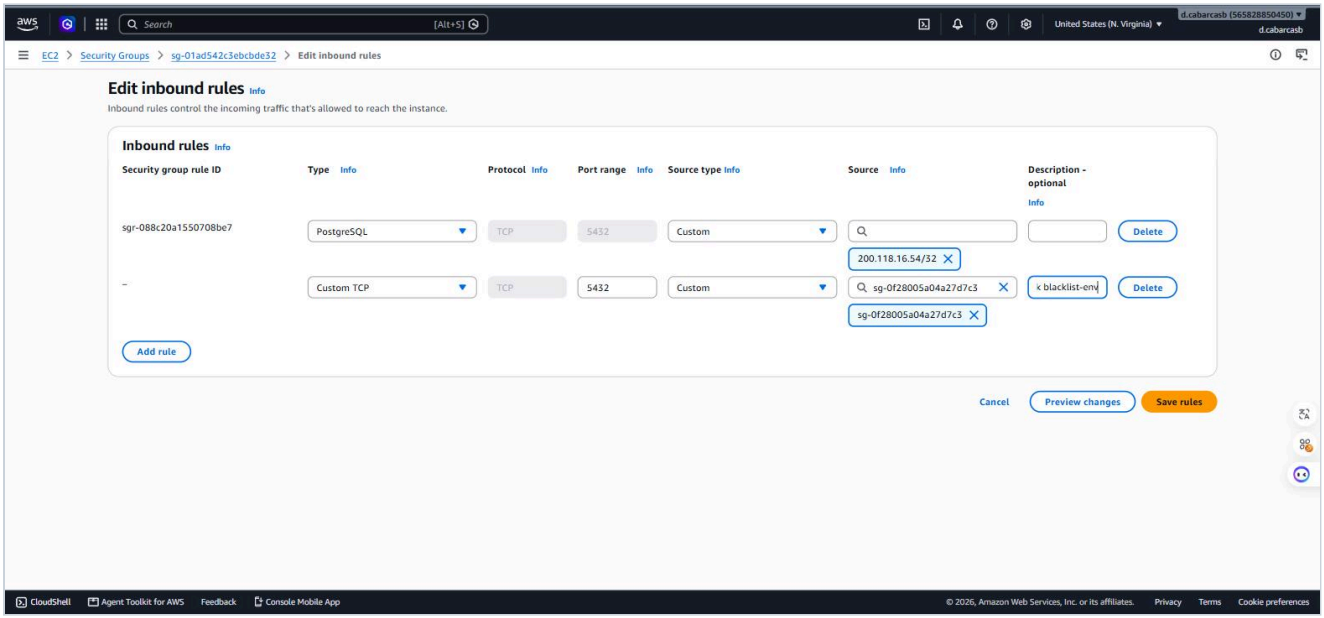


Figura 9. Regla de entrada PostgreSQL desde el security group de Beanstalk.

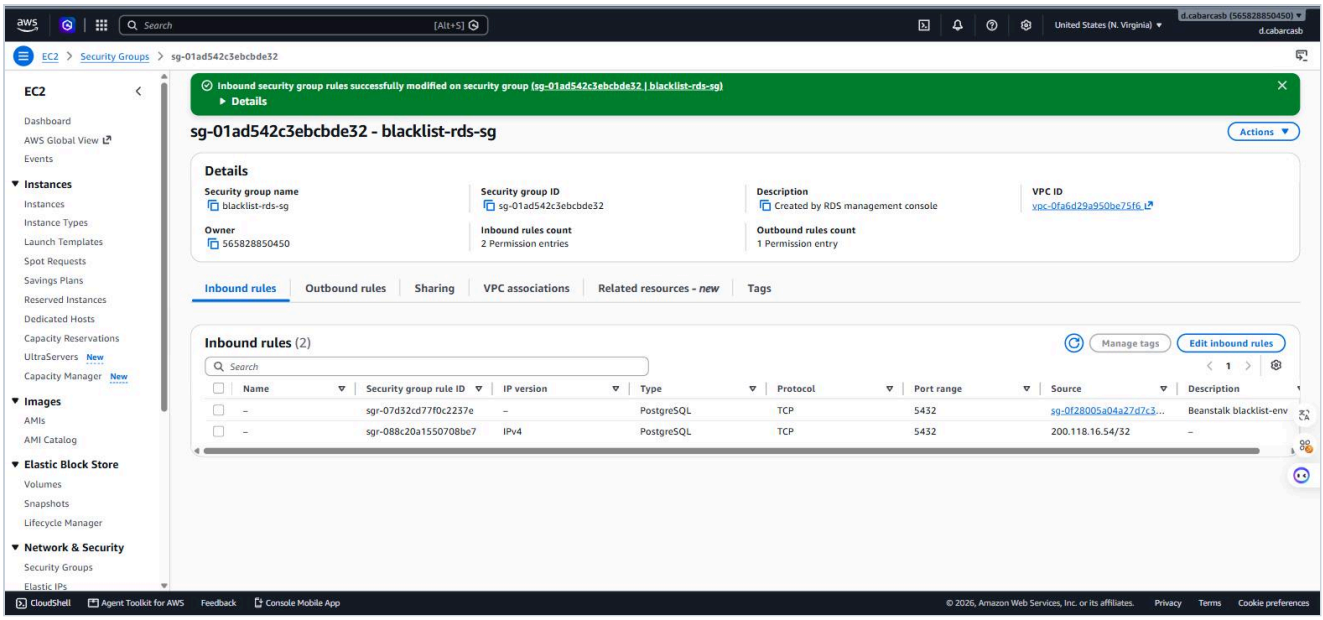


Figura 10. Reglas de entrada guardadas en blacklist-rds-sg.

1.b Configuración del proyecto en AWS Elastic Beanstalk

El repositorio incluye lo necesario para la plataforma Python: un `Procfile` que levanta gunicorn y el archivo `.ebextensions/01_environment.config` con el tipo de ambiente, el Auto Scaling de 3 a 6 instancias, el health check y el reporte de salud mejorado. El paquete se generó como zip desde el repositorio y se cargó desde la consola (*Create environment*).

Parámetro	Valor
Aplicación / ambiente	`blacklist-app` / `blacklist-env`
Dominio	`blacklist-grupo16.us-east-1.elasticbeanstalk.com`
Plataforma	Python 3.12 on 64bit Amazon Linux 2023 (4.13.10)
Código	Archivo local `blacklist-v1.0.0.zip`, etiqueta `v1.0.0`
Variables de entorno	`DATABASE_URL` (RDS), `JWT_SECRET_KEY`, `STATIC_API_TOKEN`; valores fuera del repositorio
Roles	`aws-elasticbeanstalk-service-role` y perfil `aws-elasticbeanstalk-ec2-role`
Capacidad	Load balanced, ALB dedicado público, Auto Scaling mínimo 3 / máximo 6, `t3.micro`
Monitoreo	Enhanced health reporting
Política inicial de despliegue	All at once; actualizaciones administradas de plataforma desactivadas

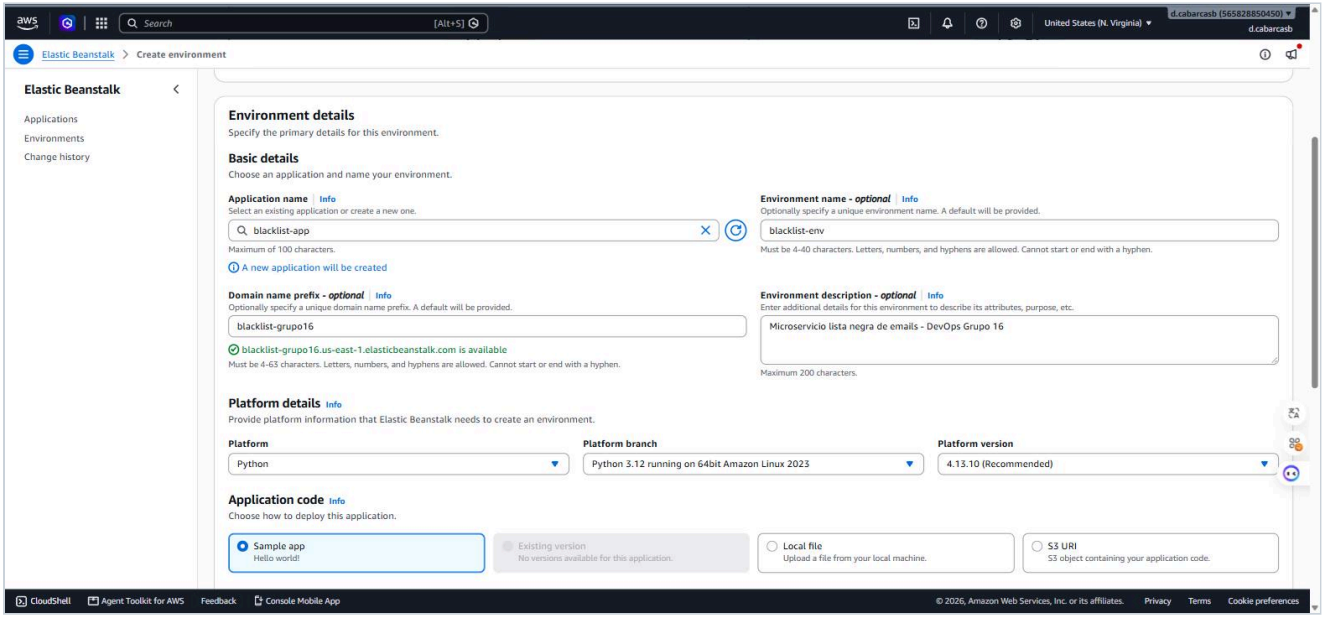


Figura 11. Aplicación, ambiente, dominio y plataforma Python 3.12.

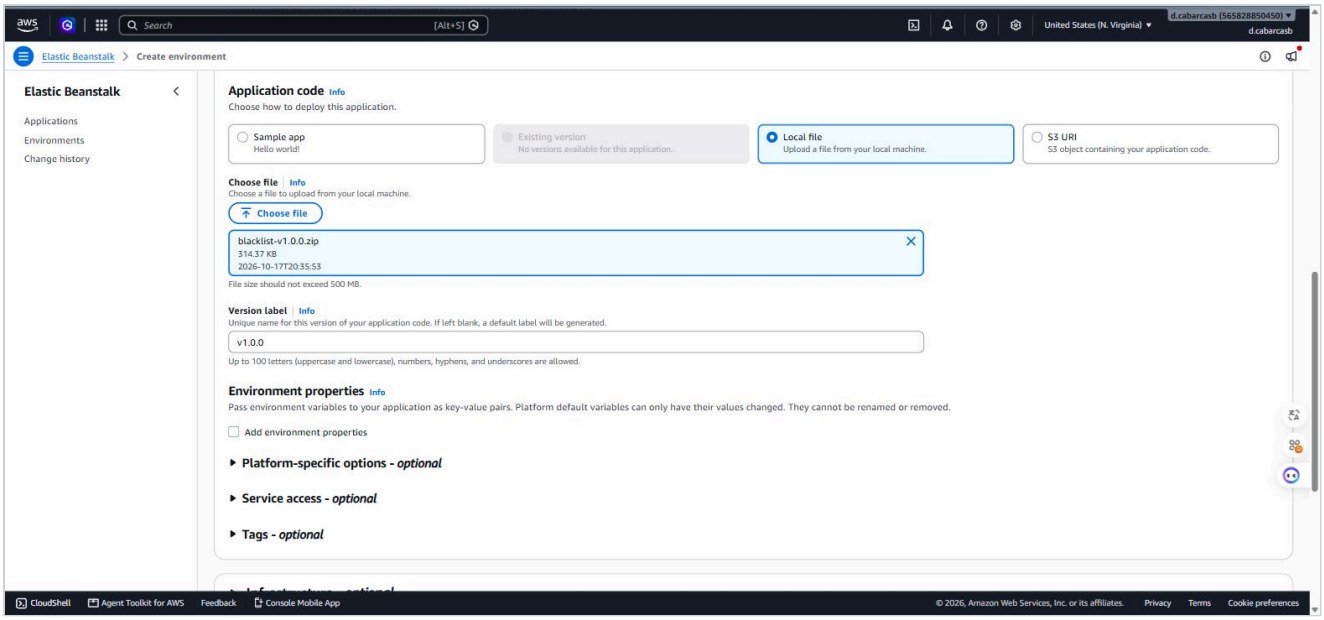


Figura 12. Carga del código fuente (zip) con la etiqueta v1.0.0.

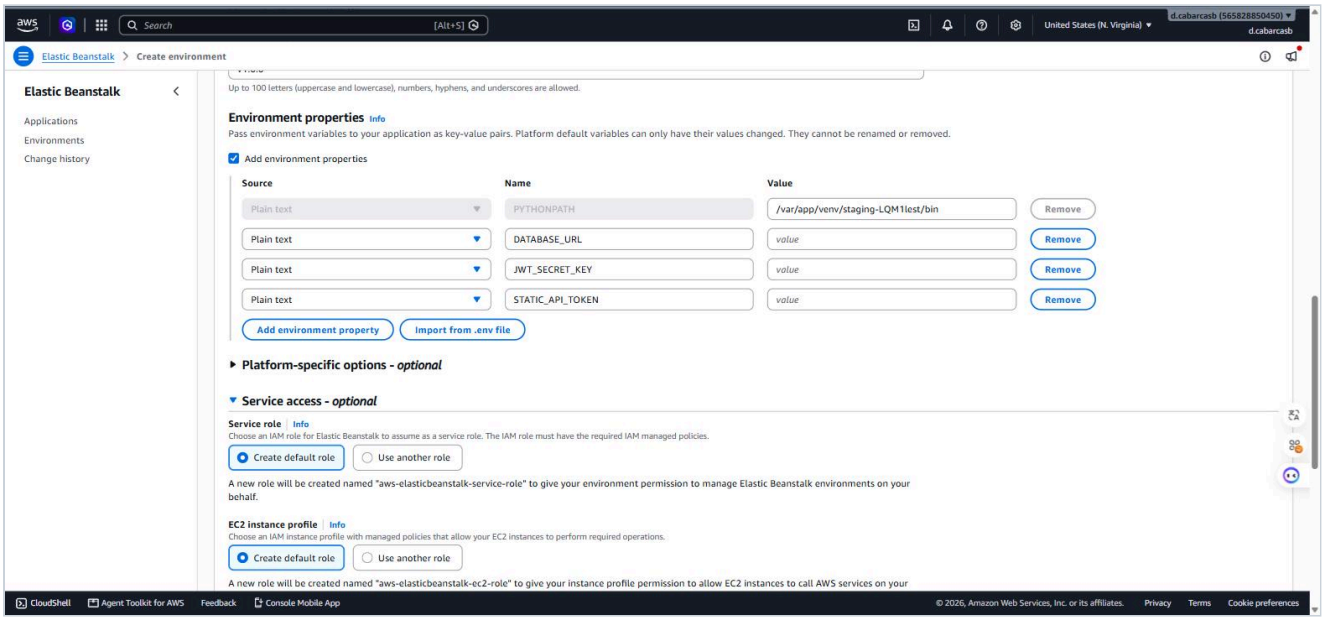


Figura 13. Variables de entorno (valores omitidos) y roles de servicio.

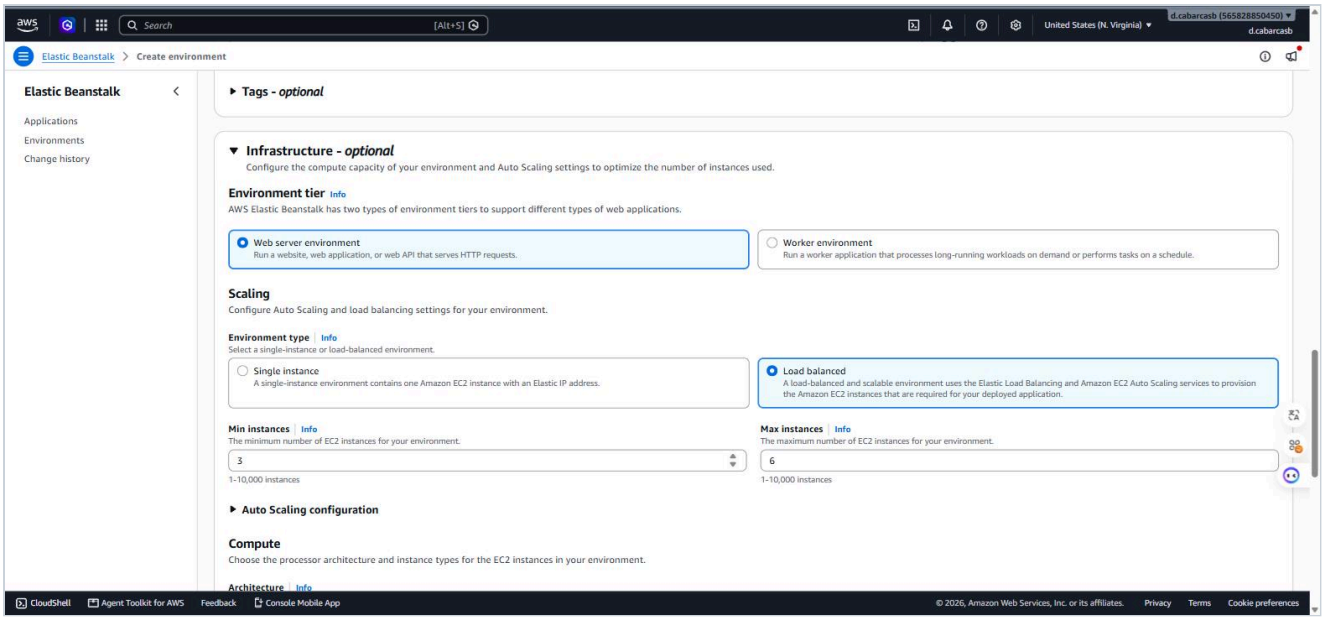


Figura 14. Ambiente con balanceo de carga y Auto Scaling de 3 a 6 instancias.

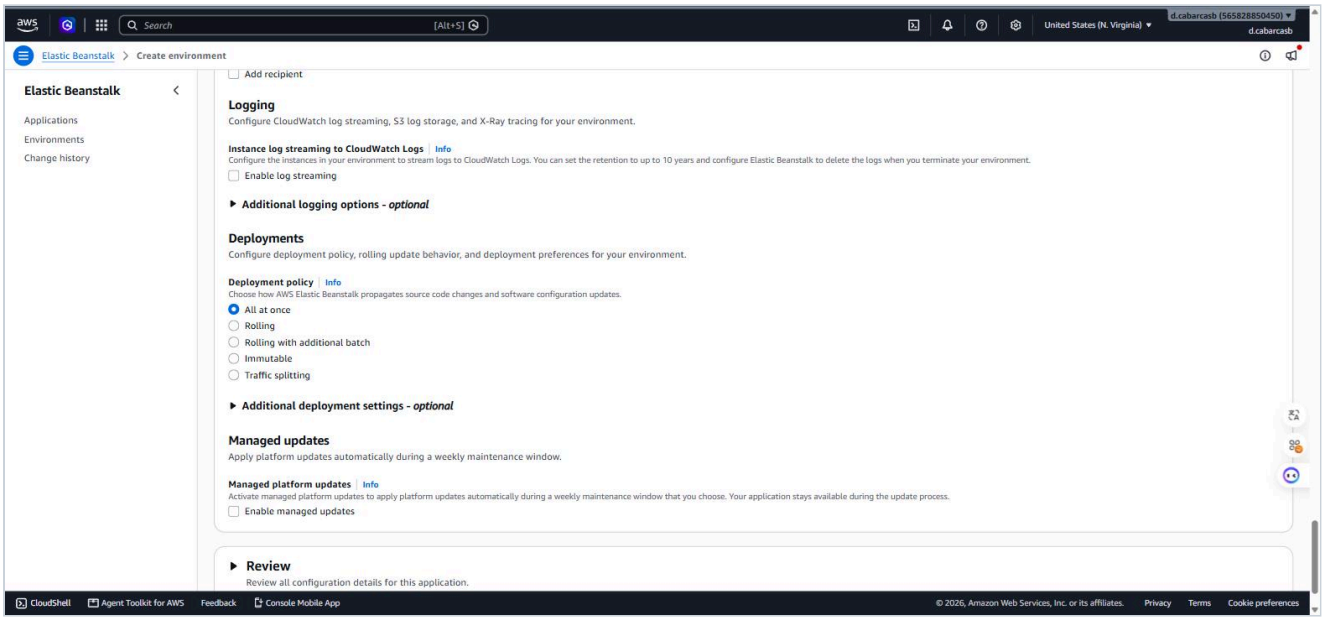


Figura 15. Política de despliegue inicial (All at once).

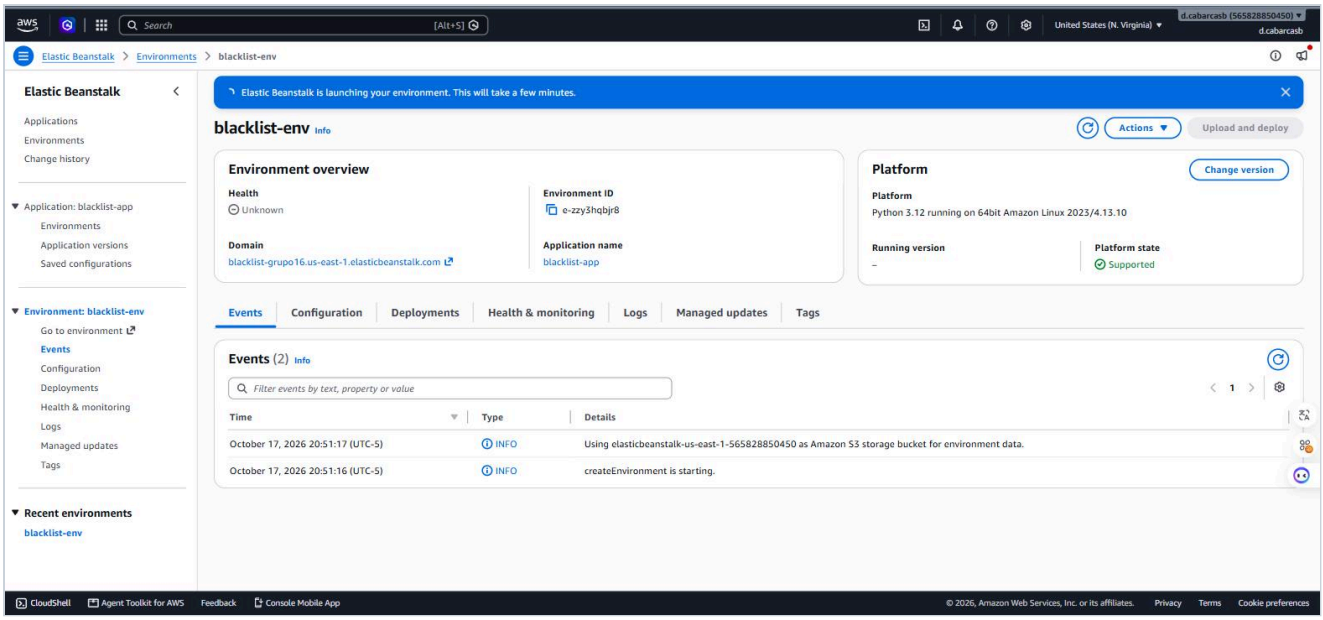


Figura 16. Lanzamiento del ambiente.

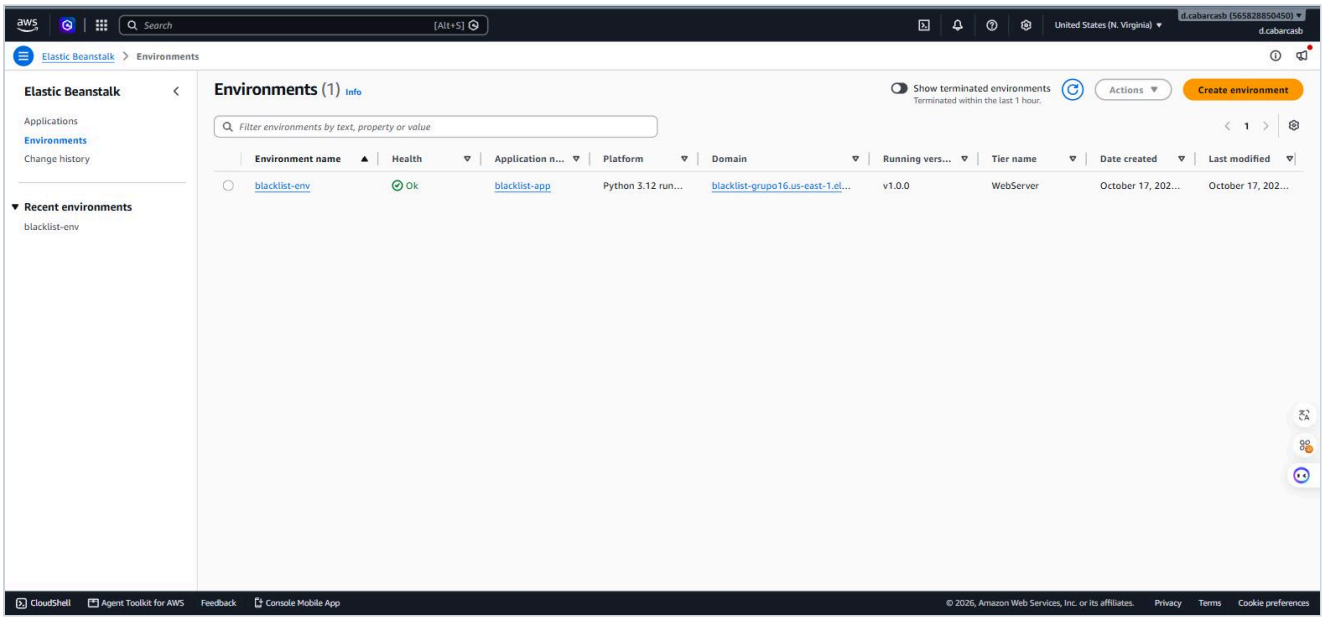


Figura 17. Ambiente blacklist-env en estado Ok con la versión v1.0.0.

El ambiente quedó disponible en unos 3 minutos con 3 instancias. Luego se validaron los endpoints contra AWS:

Escenario	Resultado
POST /blacklists con email nuevo	201 Created
POST /blacklists con email duplicado	409 Conflict
POST /blacklists con UUID inválido	400 Bad Request
POST /blacklists sin token	401 Unauthorized
GET /blacklists/<email bloqueado>	200 `{"is_blacklisted": true, "blocked_reason": "Prueba desde AWS"}`
GET /blacklists/<email no registrado>	200 `{"is_blacklisted": false, "blocked_reason": null}`
GET /blacklists/<email> con token inválido	422 Unprocessable Entity

1.c Configuración de los health checks en AWS Elastic Beanstalk

Se expuso el endpoint `GET /health`, que no requiere token ni consulta la base de datos y responde `200` con el estado, la versión desplegada y el host que atendió la solicitud. En el proceso *default* del Application Load Balancer se configuró:

Parámetro	Valor
Health check path	`/health`
Código HTTP esperado	200
Intervalo / timeout	15 s / 5 s
Healthy threshold / Unhealthy threshold	3 / 5
Deregistration delay	20 s
Health reporting	Enhanced

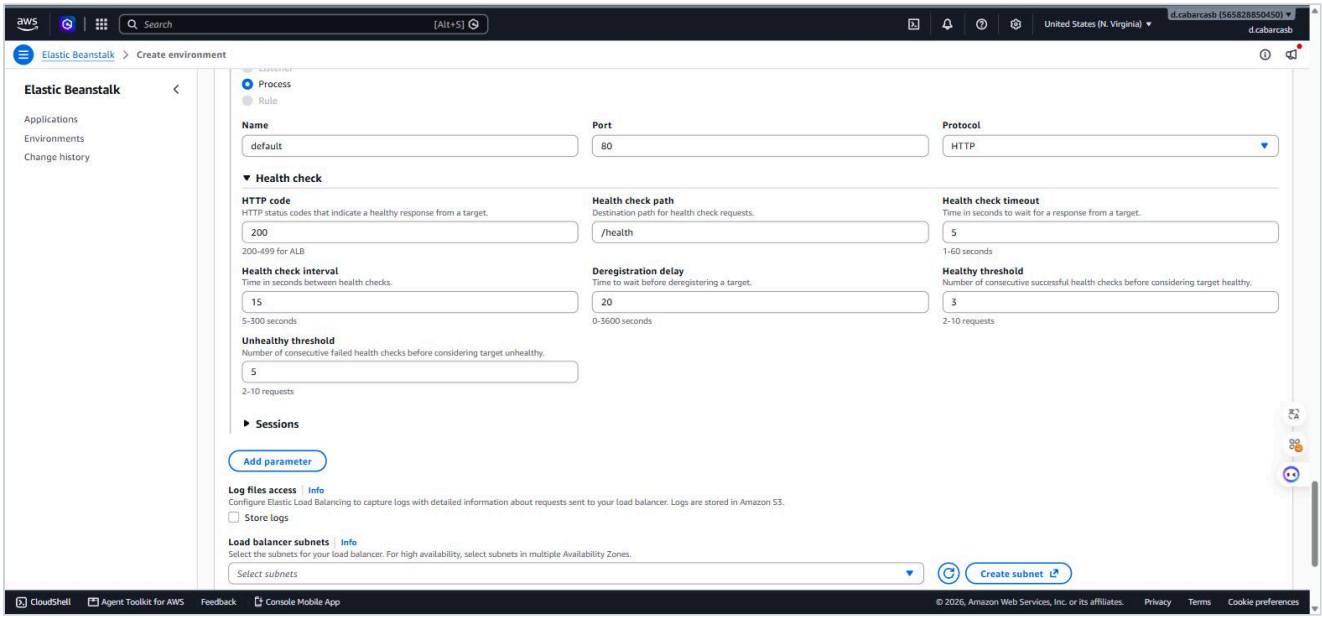


Figura 18. Health check del proceso default del ALB en /health.

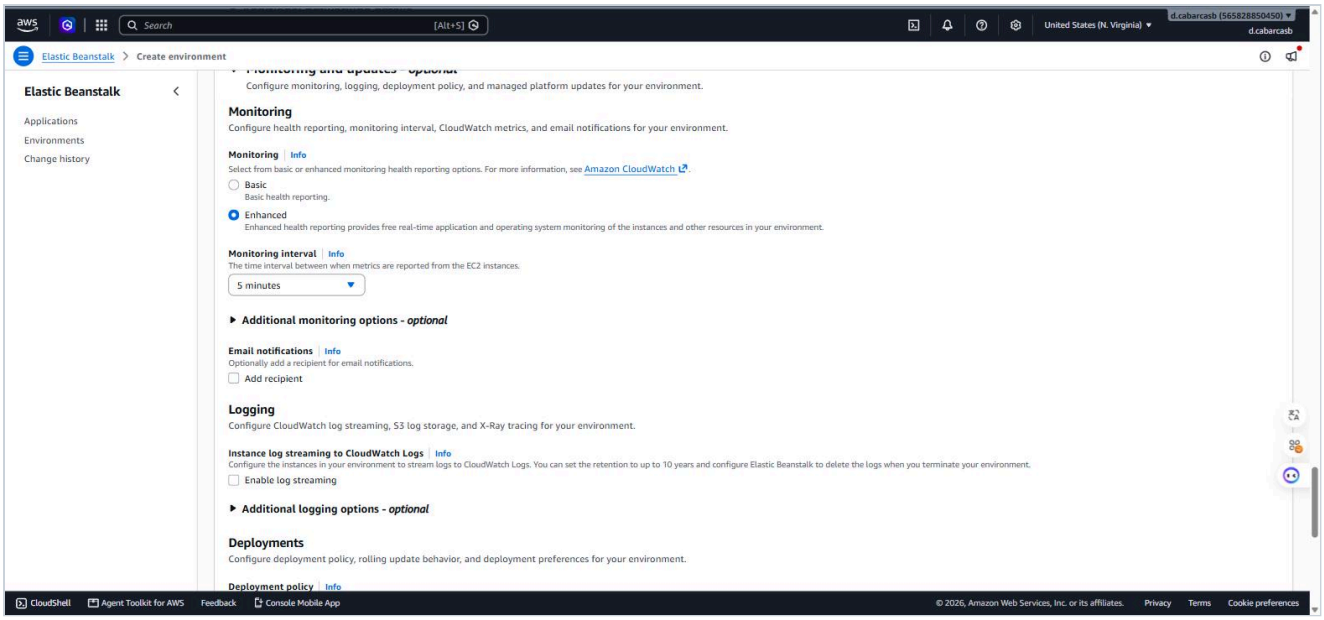


Figura 19. Reporte de salud mejorado (Enhanced).

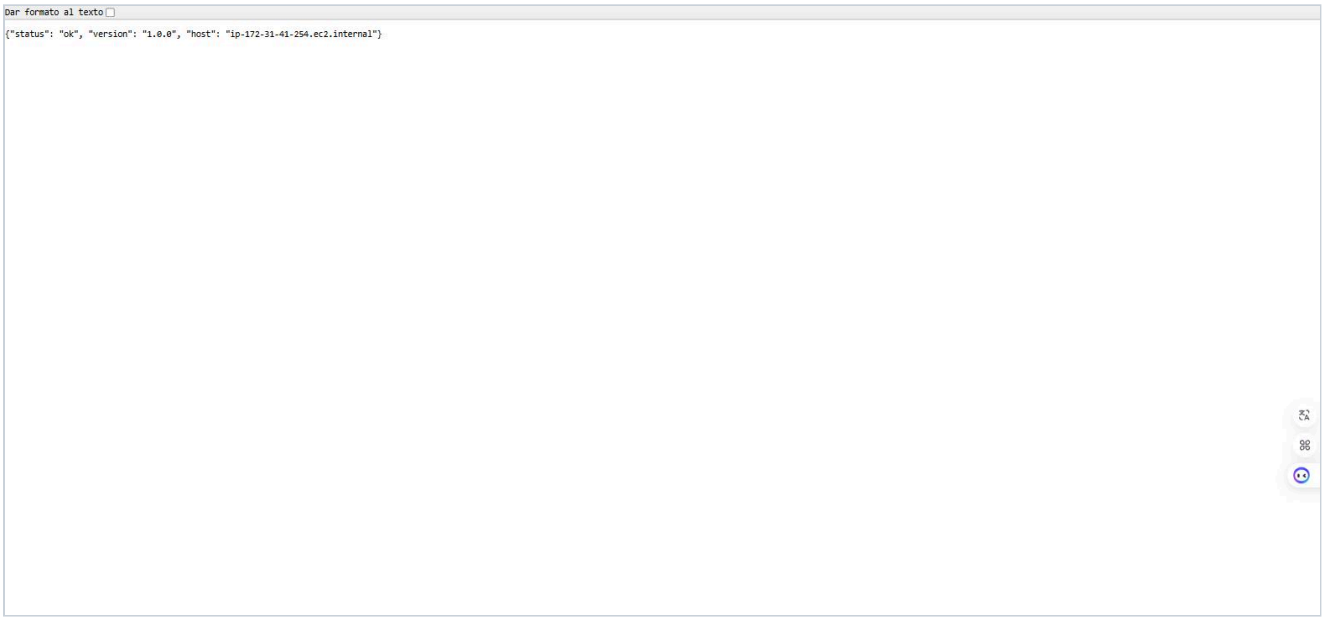


Figura 20. Respuesta del endpoint /health a través del balanceador.

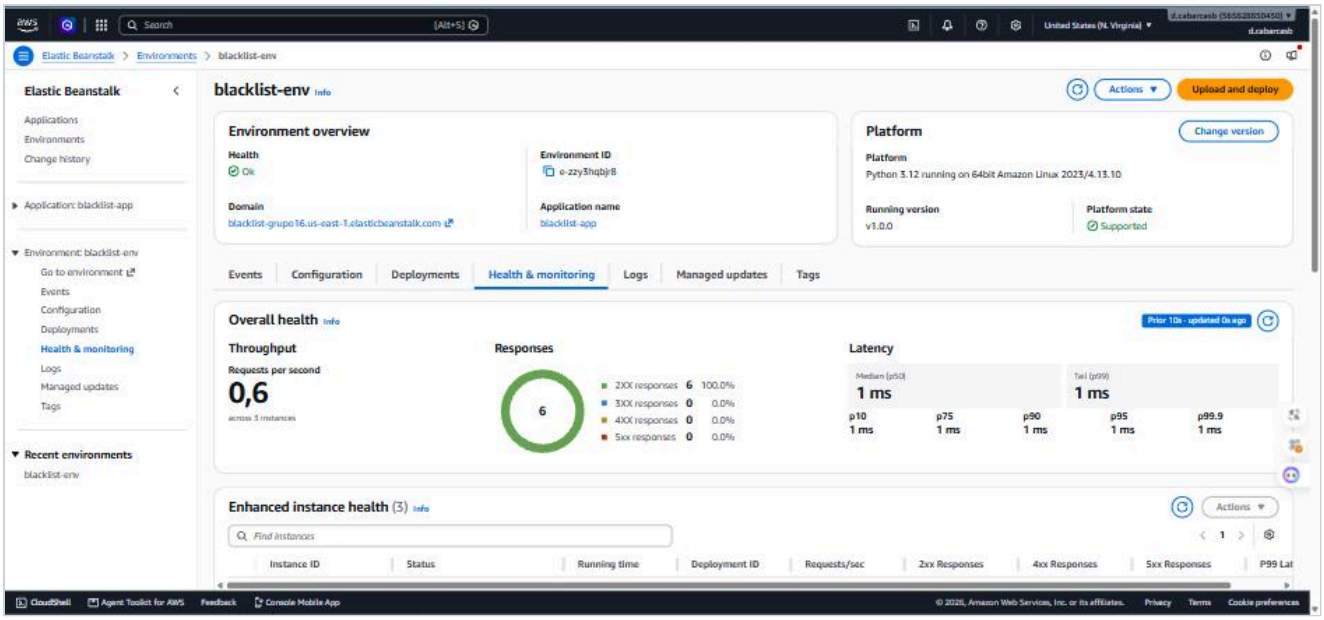


Figura 21. Salud general del ambiente: 100 % de respuestas 2xx.

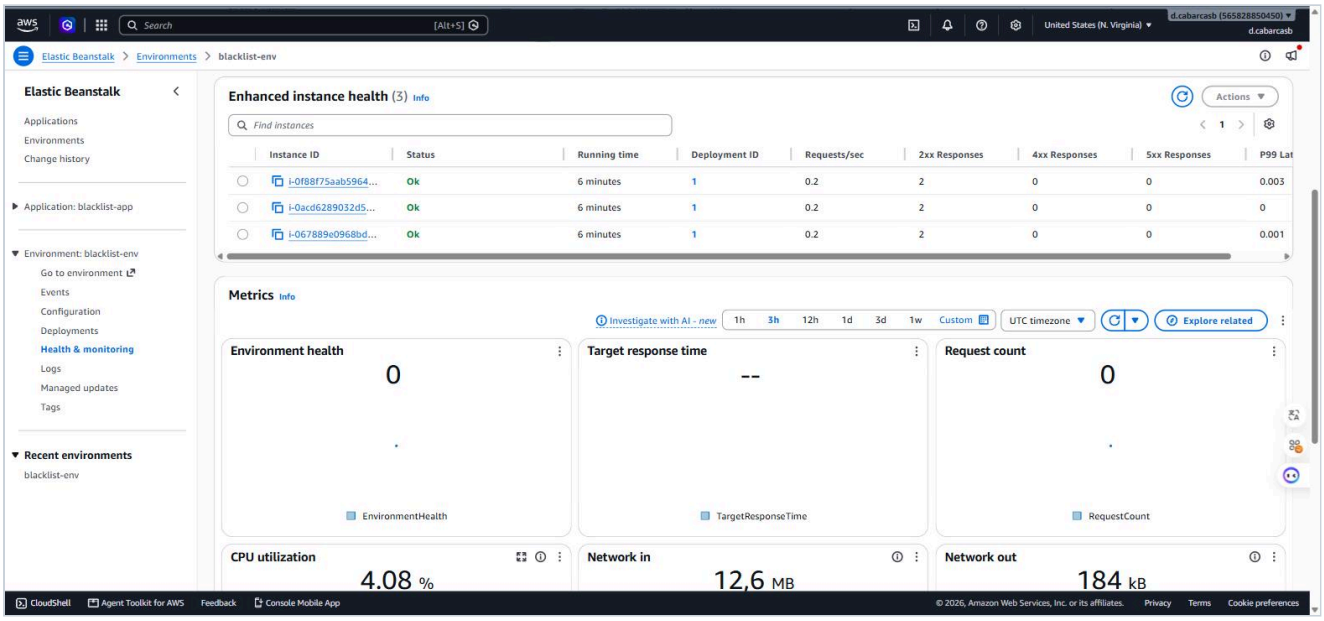


Figura 22. Enhanced instance health: las 3 instancias en estado Ok.

2. Estrategias de despliegue

Se ejecutaron cuatro estrategias que requieren más de una instancia, sobre el Auto Scaling group de 3 a 6 instancias. Cada estrategia desplegó una versión nueva del mismo código (v1.0.1 a v1.0.4). La versión viaja en el archivo `VERSION` del paquete y se expone en `GET /health`, lo que permite saber qué instancia y qué versión responde cada solicitud.

Método de validación (común a las cuatro estrategias)

1. Antes de desplegar se registraron los IDs de las instancias del ambiente.
2. Durante el despliegue, un monitor (`scripts/monitor-health.sh`) consultó `GET /health` cada segundo y guardó la hora, el código HTTP, la versión y la instancia que respondió (`entregas/monitoreo/*.log`).
3. En la consola se siguieron las pestañas *Events* y *Health & monitoring* (columna *Deployment ID*) y, para Immutable, EC2 → Instances.
4. El **tiempo total** se tomó de los eventos de Beanstalk, desde "*Environment update is starting.*" hasta "*Environment update completed successfully.*" (UTC).
5. Al terminar se compararon los IDs de las instancias con los iniciales.

Resumen

#	Estrategia	Versión	a. Instancias	c. Tiempo	d. Iniciales / nuevas	Errores observados
1	All at once	1.0.0 → 1.0.1	3	1 min 01 s	Iniciales	3 × HTTP 502 (~4 s)
2	Rolling (lote fijo 1)	1.0.1 → 1.0.2	3 (2 en servicio)	9 min 18 s	Iniciales	0 errores HTTP
3	Rolling with additional batch (lote 1)	1.0.2 → 1.0.3	3 → 4 → 3	10 min 26 s	Mixto (1 nueva + 2 iniciales)	0 errores HTTP
4	Immutable	1.0.3 → 1.0.4	3 → 6 → 3	18 min 06 s	Nuevas	0 errores HTTP

2.1 All at once (v1.0.0 → v1.0.1)

a. Instancias utilizadas	3. Las tres instancias del Auto Scaling group se actualizan simultáneamente; no se lanzan instancias adicionales.
b. Validación	Monitor de `/health` (147 peticiones) y Health & monitoring, donde las 3 instancias pasaron juntas al *Deployment ID 2*. La última respuesta 1.0.0 y la primera 1.0.1 estuvieron separadas por ~6 s sin respuestas válidas; nunca convivieron ambas versiones.
c. Tiempo total	**1 min 01 s** (02:21:03.9 → 02:22:04.6 UTC)
d. Instancias iniciales o nuevas	**Iniciales**: `i-0acd6289032d5be16`, `i-0f88f75aab5964f23`, `i-067889e0968bd9361`, las mismas antes y después.

e. Capturas

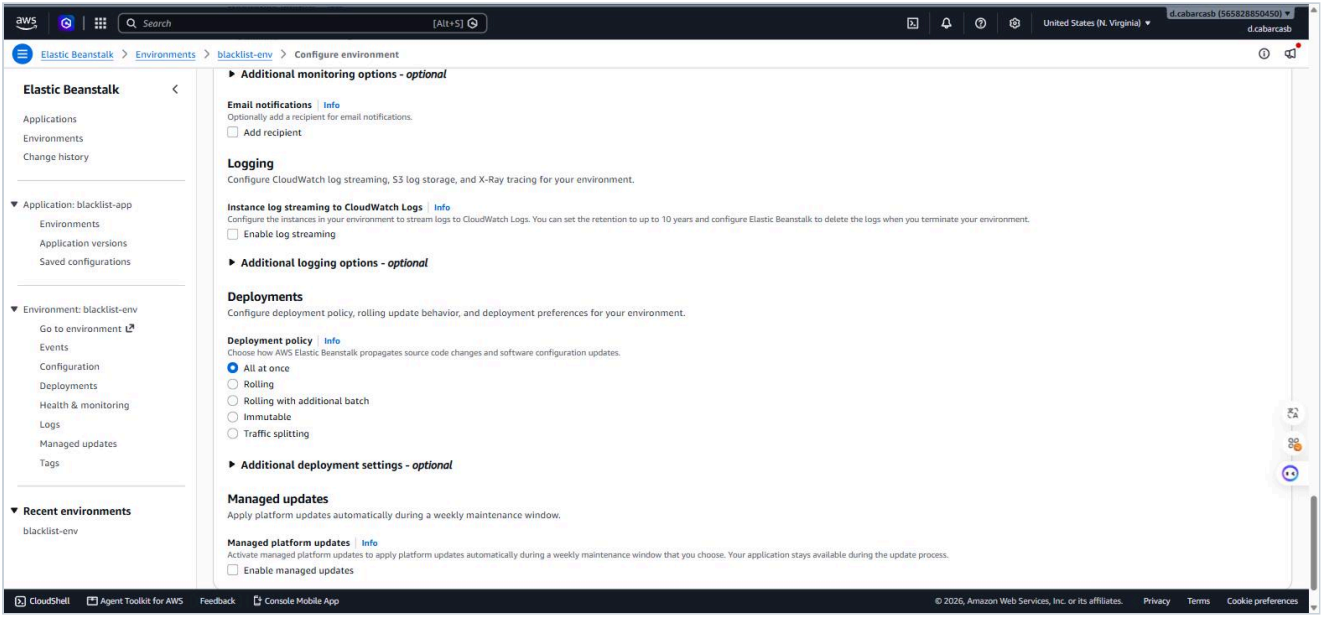


Figura 23. Política de despliegue All at once.

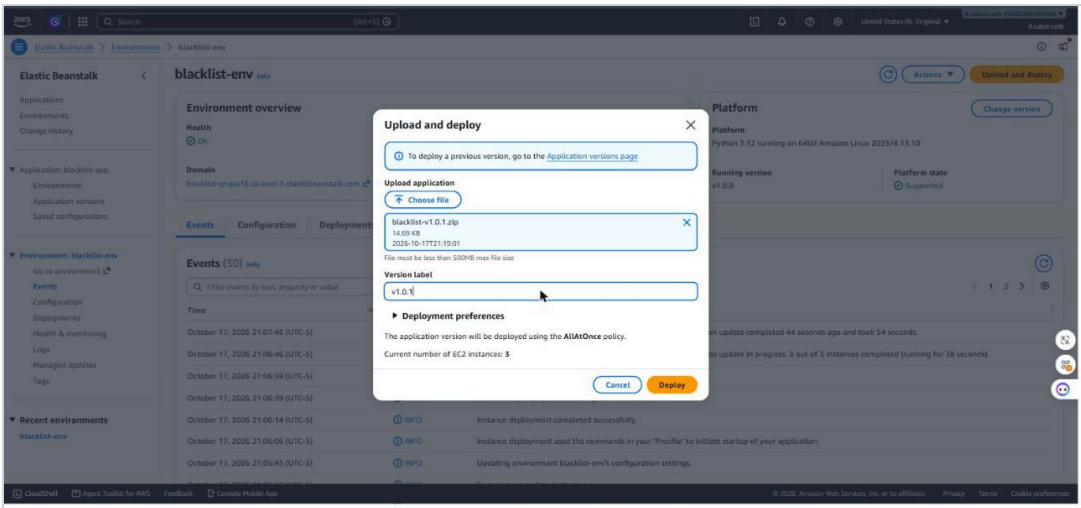


Figura 24. Carga y despliegue de la versión v1.0.1 (política AllAtOnce, 3 instancias).

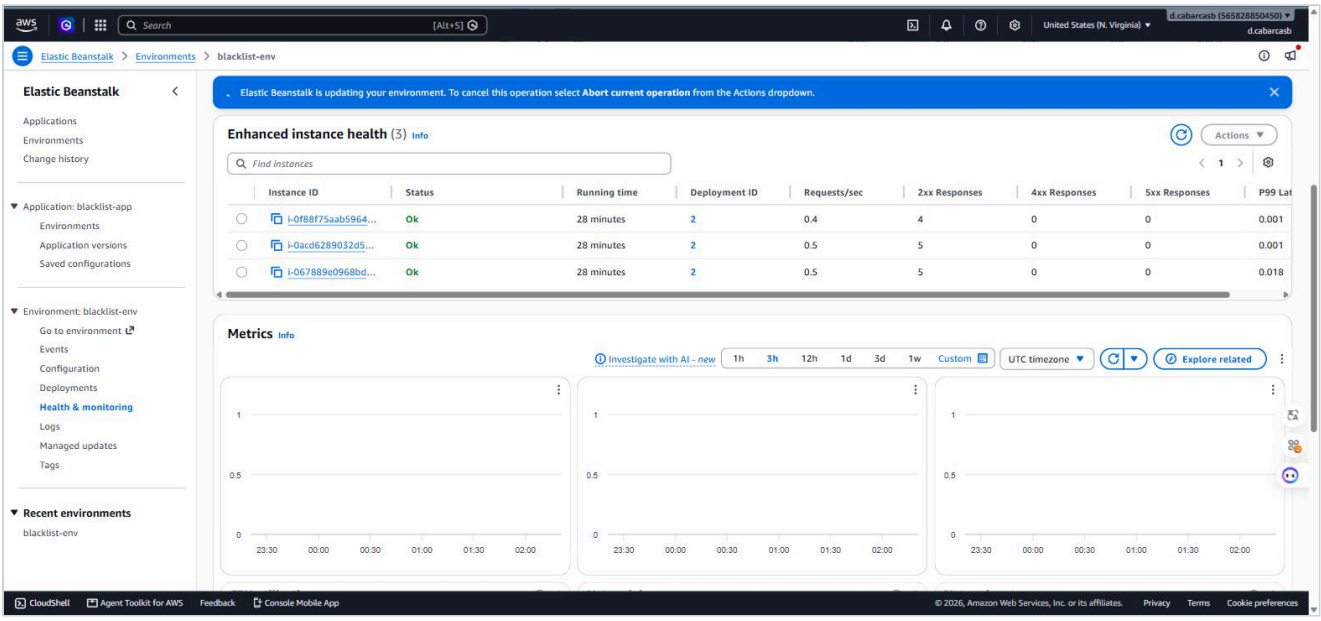


Figura 25. Las 3 instancias con el Deployment ID 2 al mismo tiempo.

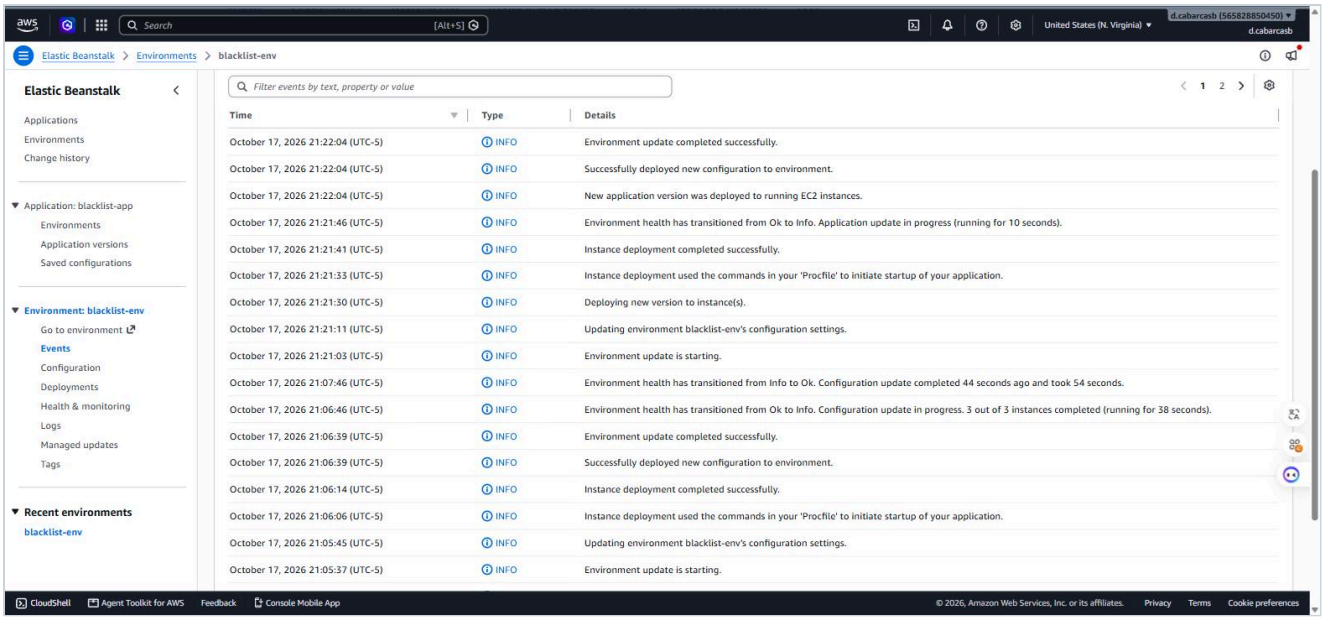


Figura 26. Eventos del despliegue, de inicio a fin.

f. Hallazgos

- Es la estrategia más rápida, pero **interrumpe el servicio**: hubo 3 respuestas `502 Bad Gateway` consecutivas (~4 s) mientras gunicorn se reiniciaba en todas las instancias a la vez.
- No hay mezcla de versiones: desde el punto de vista del cliente el cambio es atómico.
- No necesita capacidad adicional, pero un error en la versión nueva afectaría al 100 % de las instancias, y el rollback implica otro despliegue completo.

2.2 Rolling, lote fijo de 1 instancia (v1.0.1 → v1.0.2)

a. Instancias utilizadas	3. Se actualiza 1 instancia por lote (3 lotes); en cada lote solo 2 instancias atienden tráfico. No se lanzan instancias nuevas.
b. Validación	Monitor de `/health` (514 peticiones), eventos "*Batch N: Starting application deployment on instance(s)*" y Health & monitoring, donde se ve una instancia en *Pending application update* y luego con el *Deployment ID 4* mientras las otras seguían en el 3. En la ventana de mezcla el balanceador devolvió 106 respuestas 1.0.1 y 104 respuestas 1.0.2. Orden de los lotes: `i-0acd6289032d5be16` → `i-0f88f75aab5964f23` → `i-067889e0968bd9361`.
c. Tiempo total	**9 min 18 s** (02:27:57.1 → 02:37:15.3 UTC)
d. Instancias iniciales o nuevas	**Iniciales** (los mismos 3 IDs antes y después).

e. Capturas

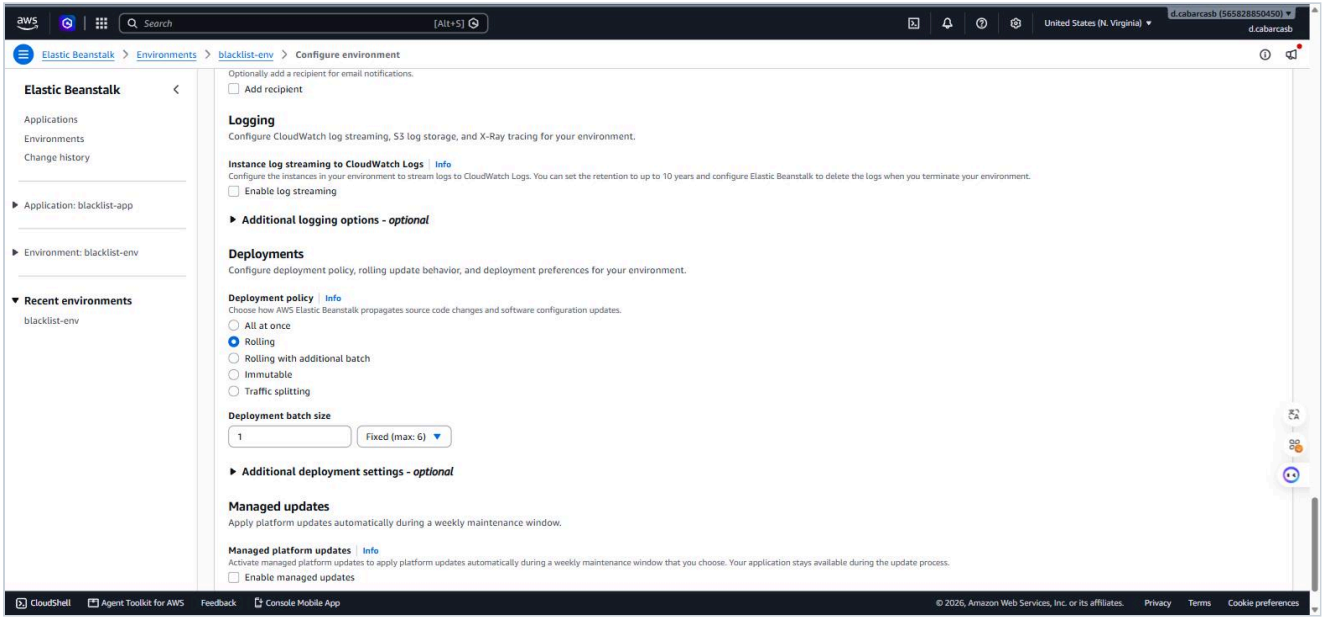


Figura 27. Política Rolling con tamaño de lote fijo de 1.

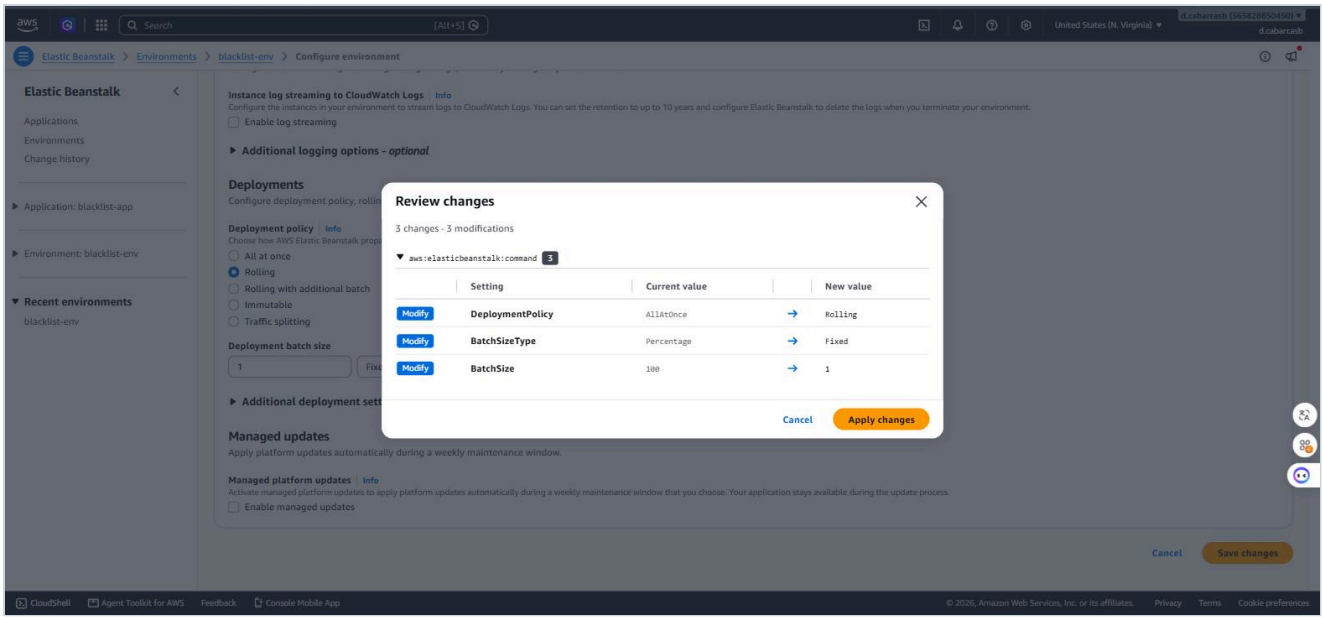


Figura 28. Revisión de cambios: AllAtOnce → Rolling, Percentage → Fixed, 100 → 1.

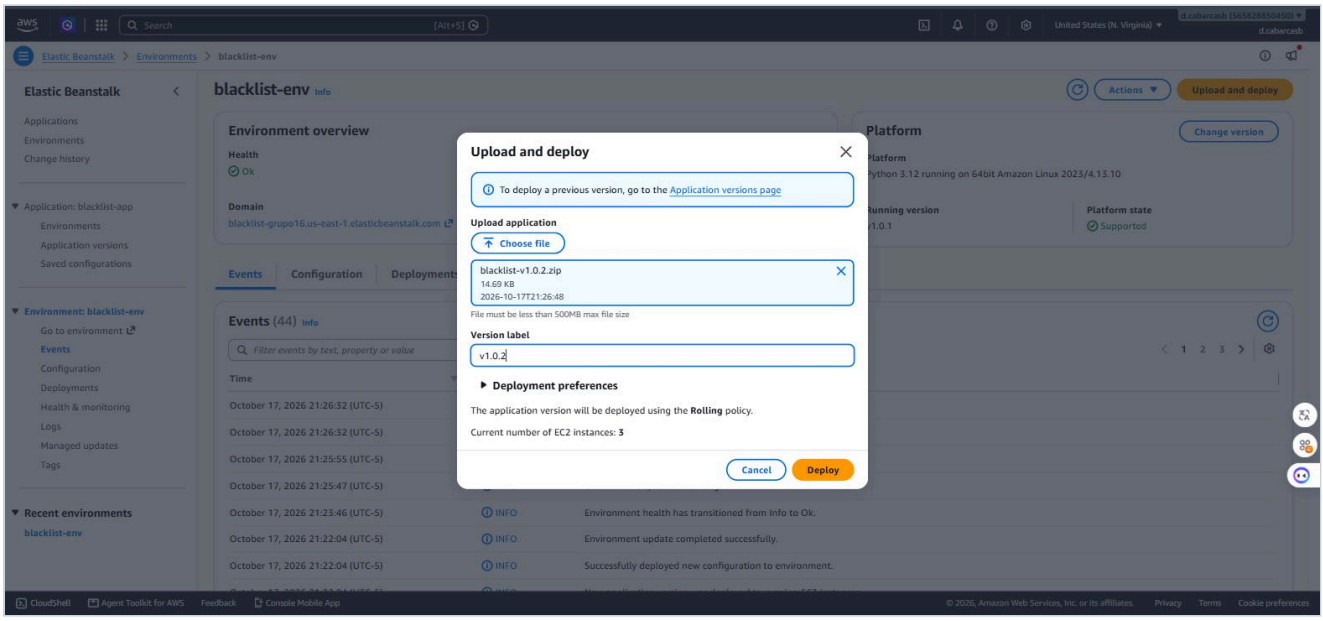


Figura 29. Despliegue de la versión v1.0.2 con la política Rolling.

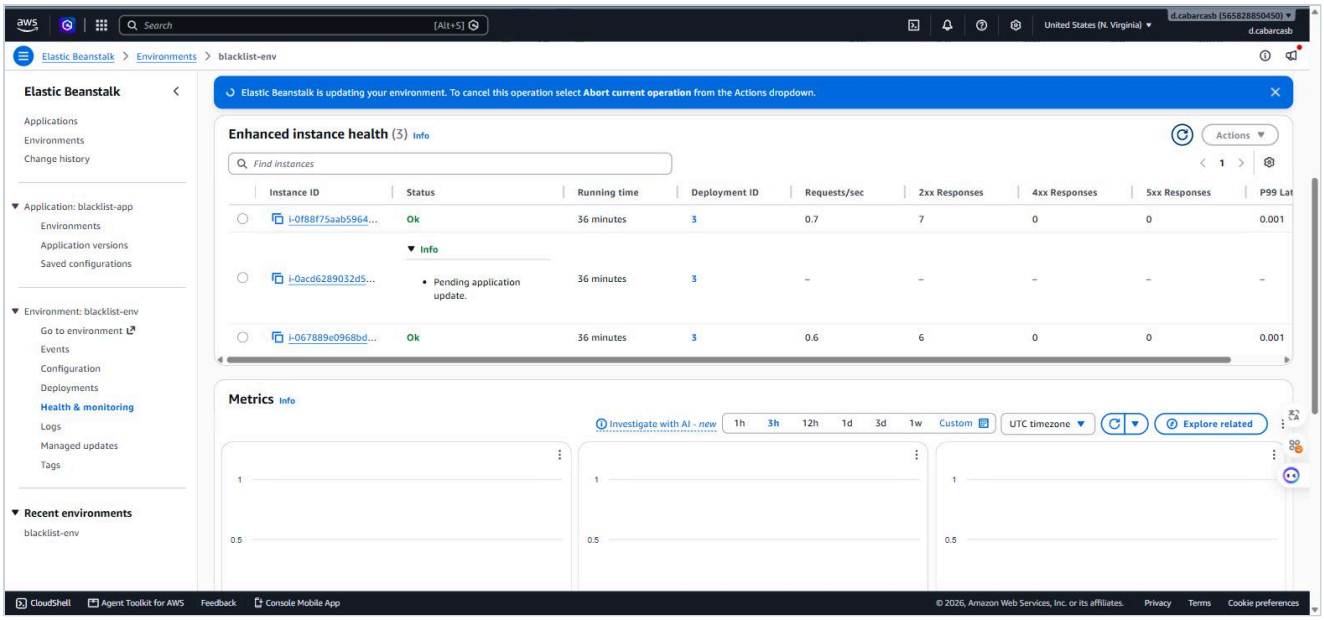


Figura 30. Lote 1: una instancia en "Pending application update" y dos atendiendo.

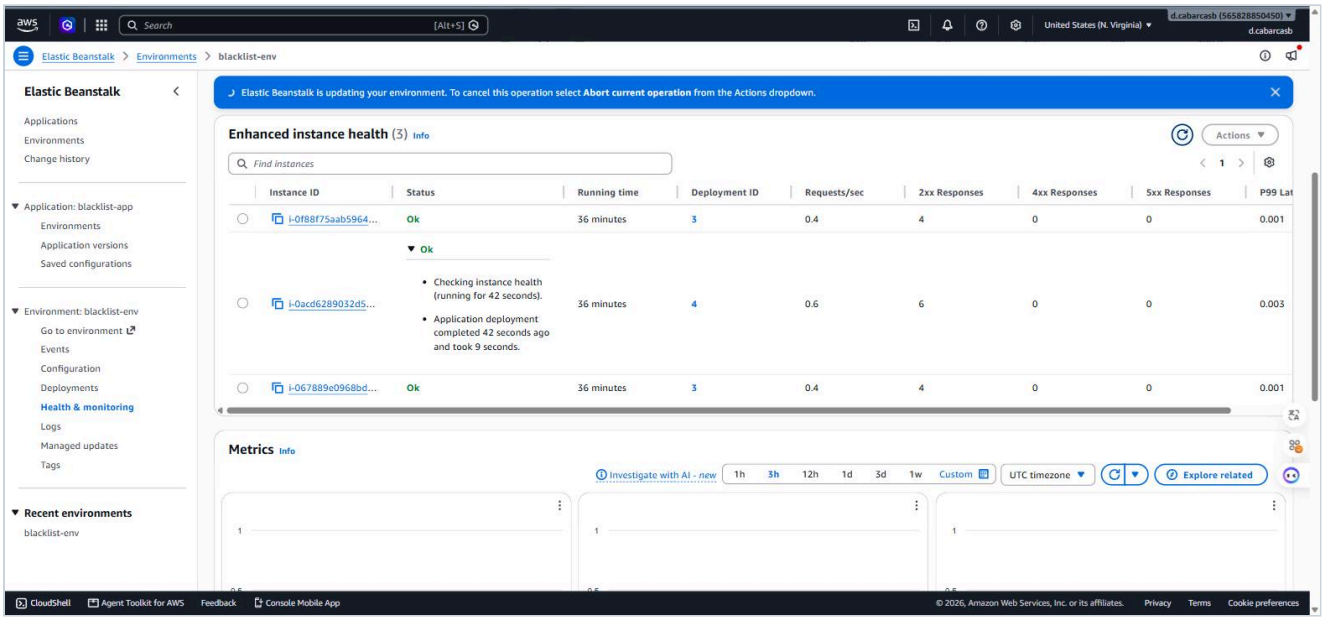


Figura 31. Versiones mezcladas: una instancia con el Deployment ID 4 y dos con el 3.

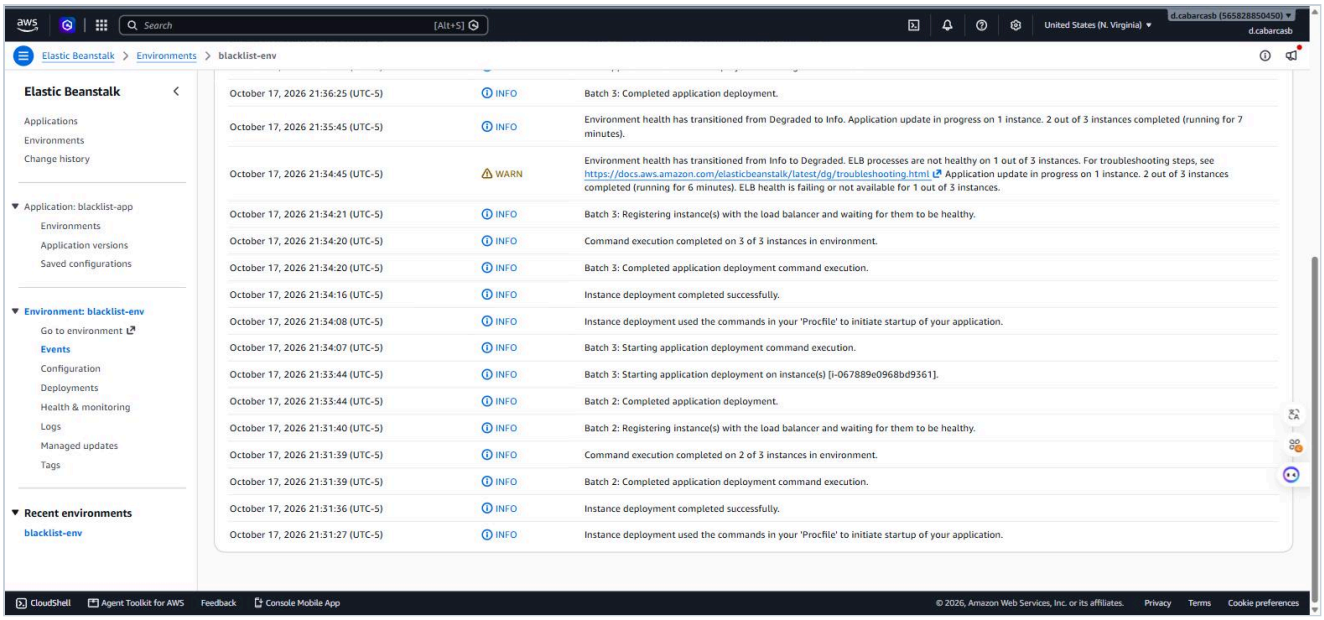


Figura 32. Eventos por lote (Batch 2 y Batch 3).

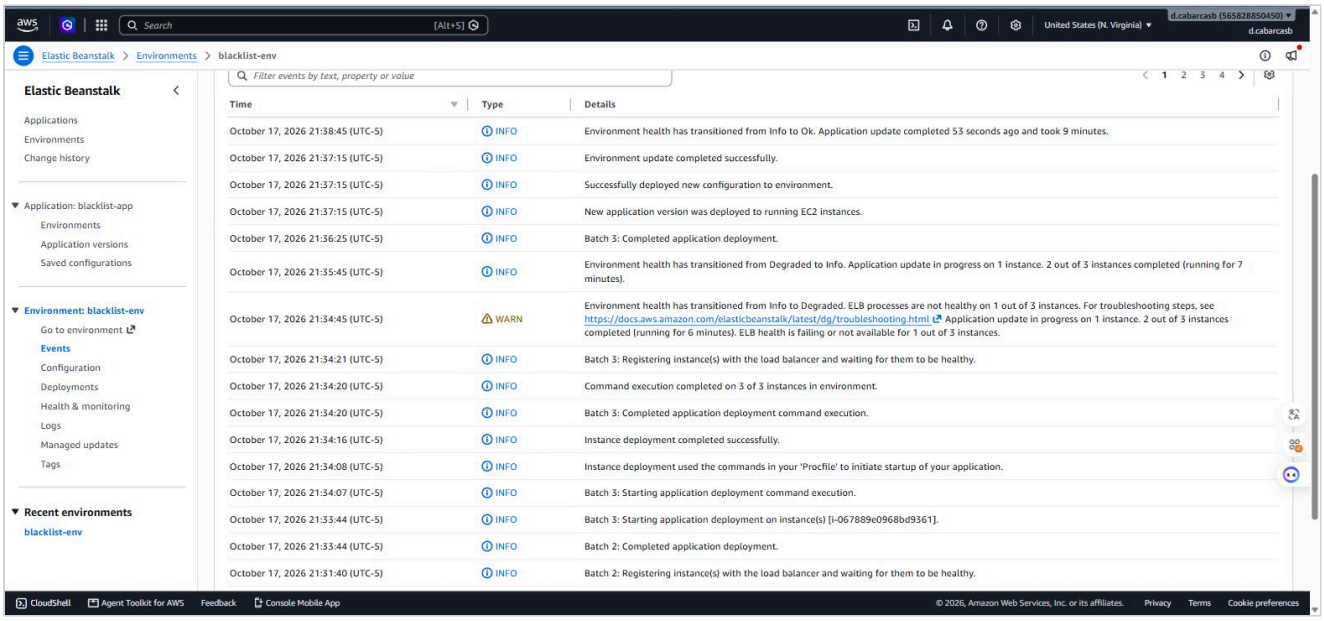


Figura 33. Finalización del despliegue ("took 9 minutes").

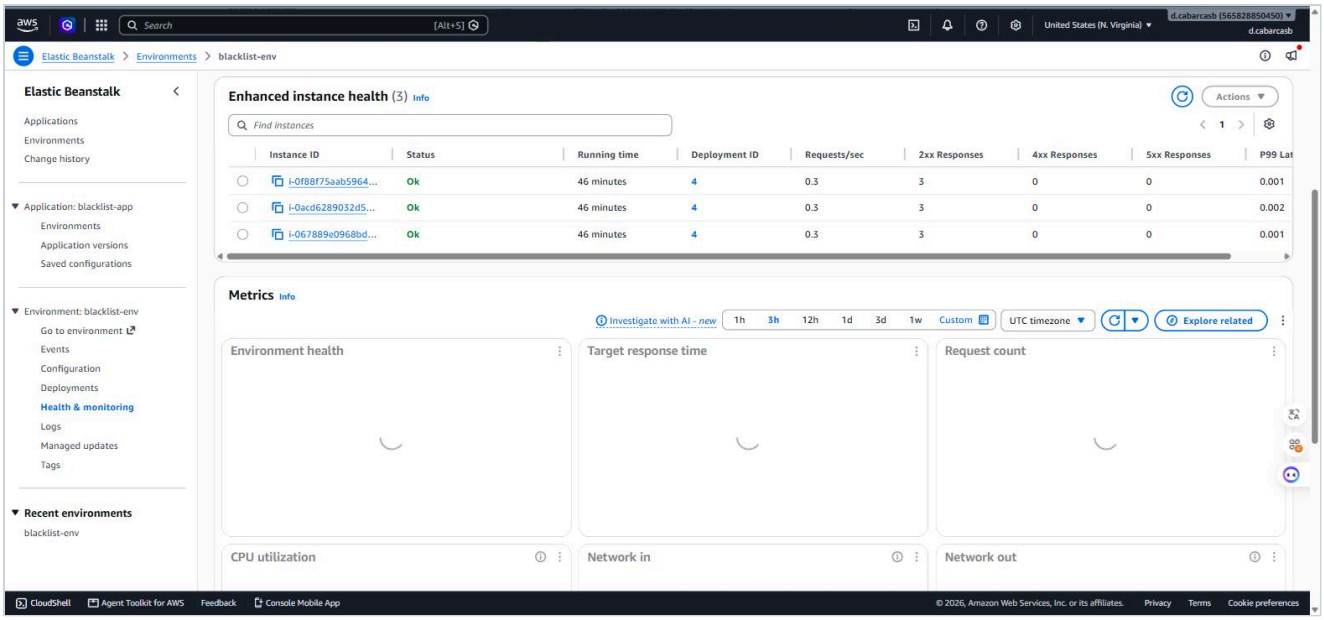


Figura 34. Estado final: las 3 instancias originales con el Deployment ID 4.

f. Hallazgos

- **Sin caída de servicio:** 0 respuestas 5xx; solo 3 timeouts aislados de 3 s (0,6 %), asociados a la salida y entrada de instancias del balanceador.
- **Capacidad reducida** al 66 % durante todo el despliegue (2 de 3 instancias).
- **Conviven dos versiones** durante ~5 min: los clientes reciben respuestas de v1.0.1 y v1.0.2 alternadamente, lo que exige que las versiones sean compatibles (API y esquema de BD).
- Es ~9 veces más lenta que All at once: cada lote tarda ~2 min 40 s, la mayor parte esperando que el ALB marque la instancia como sana (3 checks de 15 s más el desregistro). Beanstalk registró un estado *Degraded* transitorio mientras la instancia del lote 3 se registraba.

2.3 Rolling with additional batch, lote fijo de 1 (v1.0.2 → v1.0.3)

a. Instancias utilizadas	3 iniciales + **1 adicional** = **4** durante el despliegue; al final vuelven a ser 3. La capacidad en servicio nunca bajó de 3.
b. Validación	Monitor de `/health` (680 peticiones; apareció un cuarto host, `ip-172-31-88-76`, respondiendo 1.0.3), eventos y Health & monitoring con **4 instancias**: la nueva `i-0be989469ee779a06` en *Pending* y luego con el *Deployment ID 5*, mientras las originales seguían en el 4.
c. Tiempo total	**10 min 26 s** (02:44:05.7 → 02:54:32.3 UTC). El ambiente volvió a Green a las 02:56:52 UTC, tras desregistrar la instancia sobrante.
d. Instancias iniciales o nuevas	**Mixto.** Lote 1: se lanza la instancia nueva `i-0be989469ee779a06`. Lotes 2 y 3: se actualizan in-place `i-0acd6289032d5be16` e `i-067889e0968bd9361`. Al final se termina la sobrante `i-0f88f75aab5964f23`, una inicial que seguía con v1.0.2.

e. Capturas

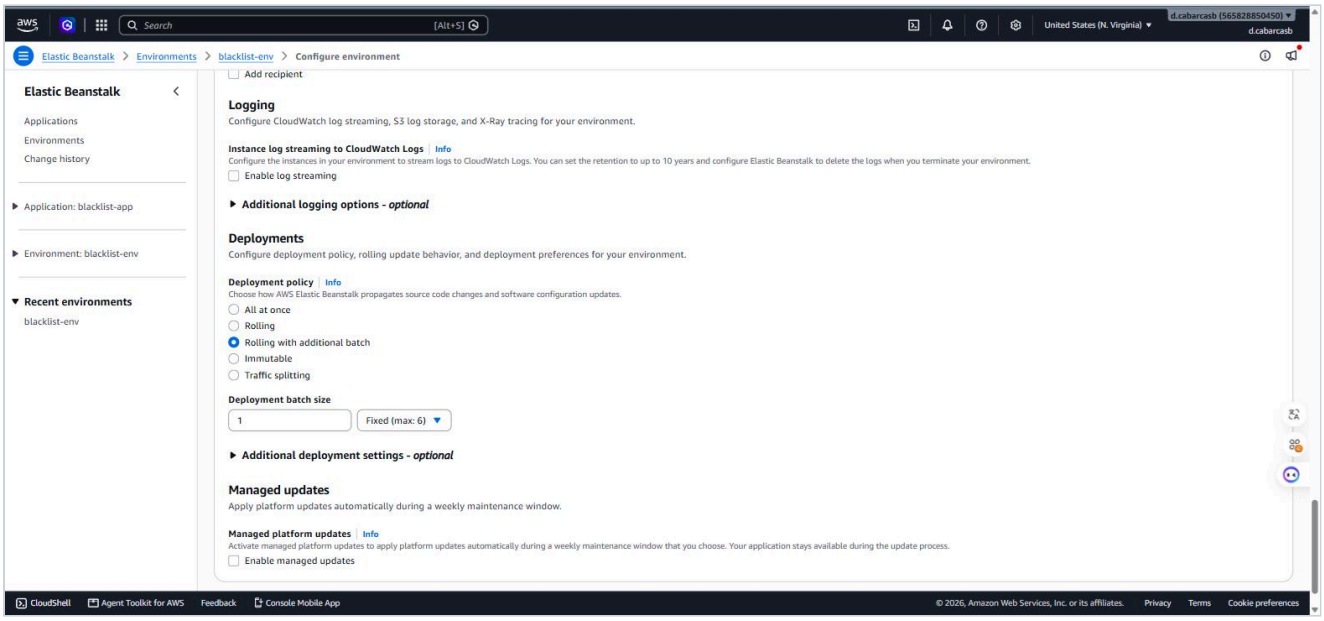


Figura 35. Política Rolling with additional batch, lote fijo de 1.

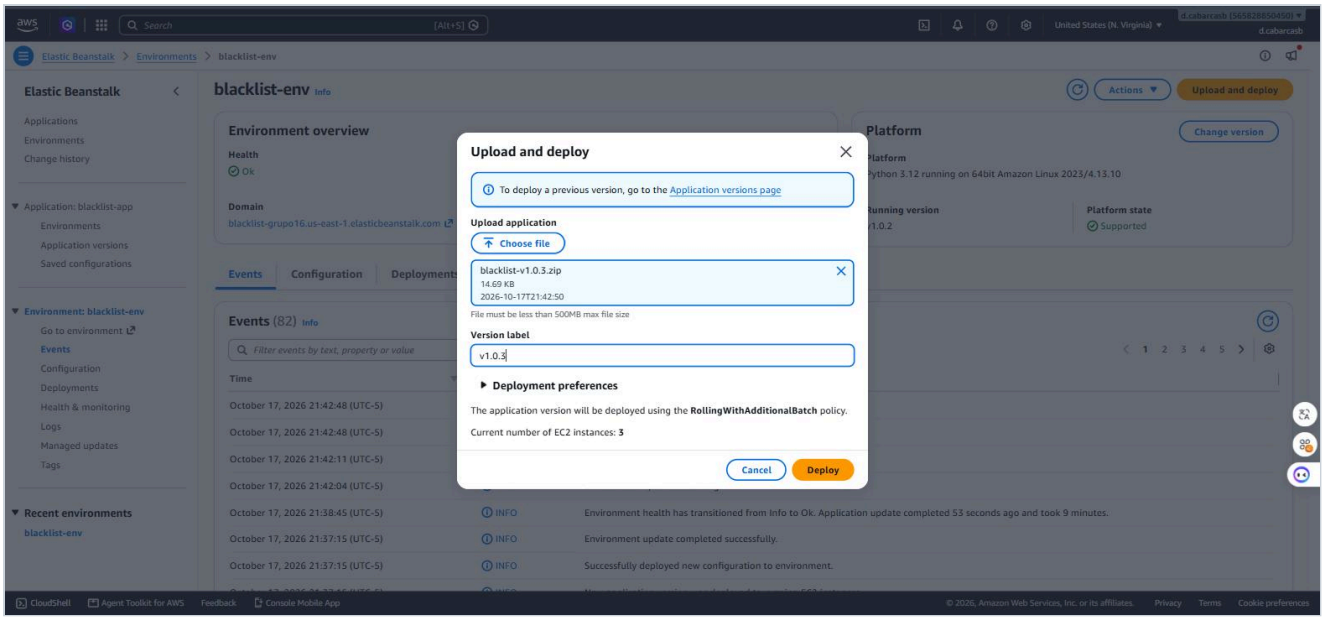


Figura 36. Despliegue de la versión v1.0.3 (RollingWithAdditionalBatch).

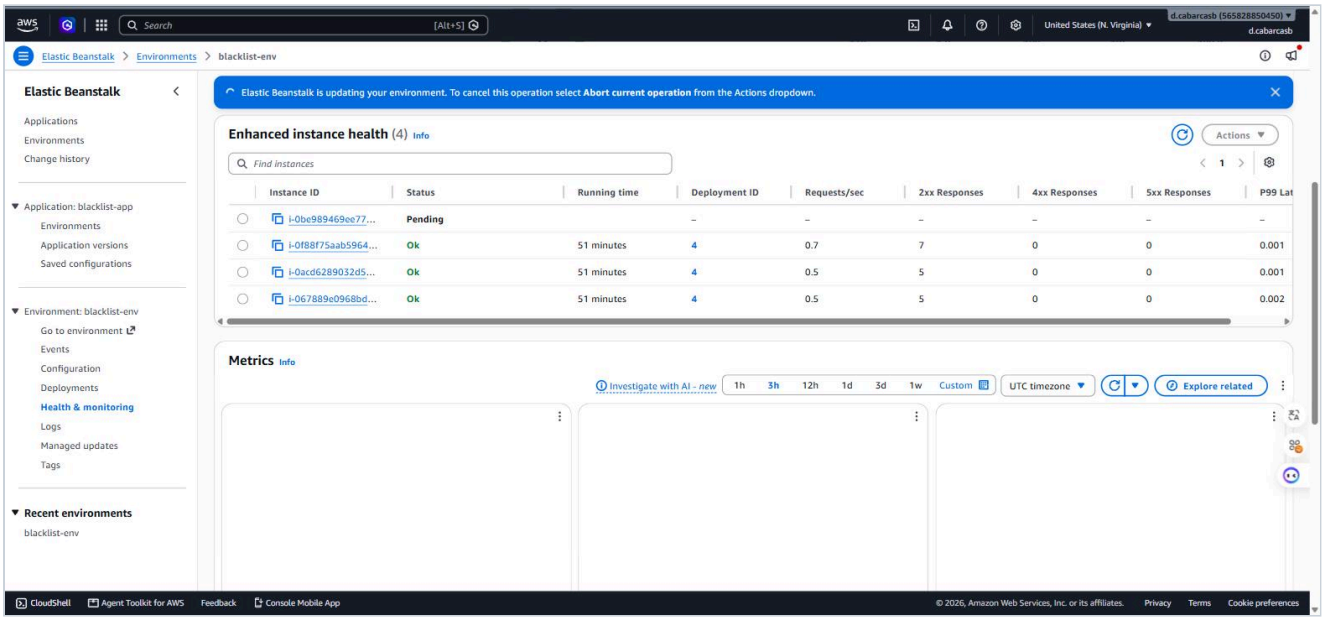


Figura 37. Lote adicional: 4 instancias, la nueva en estado Pending.

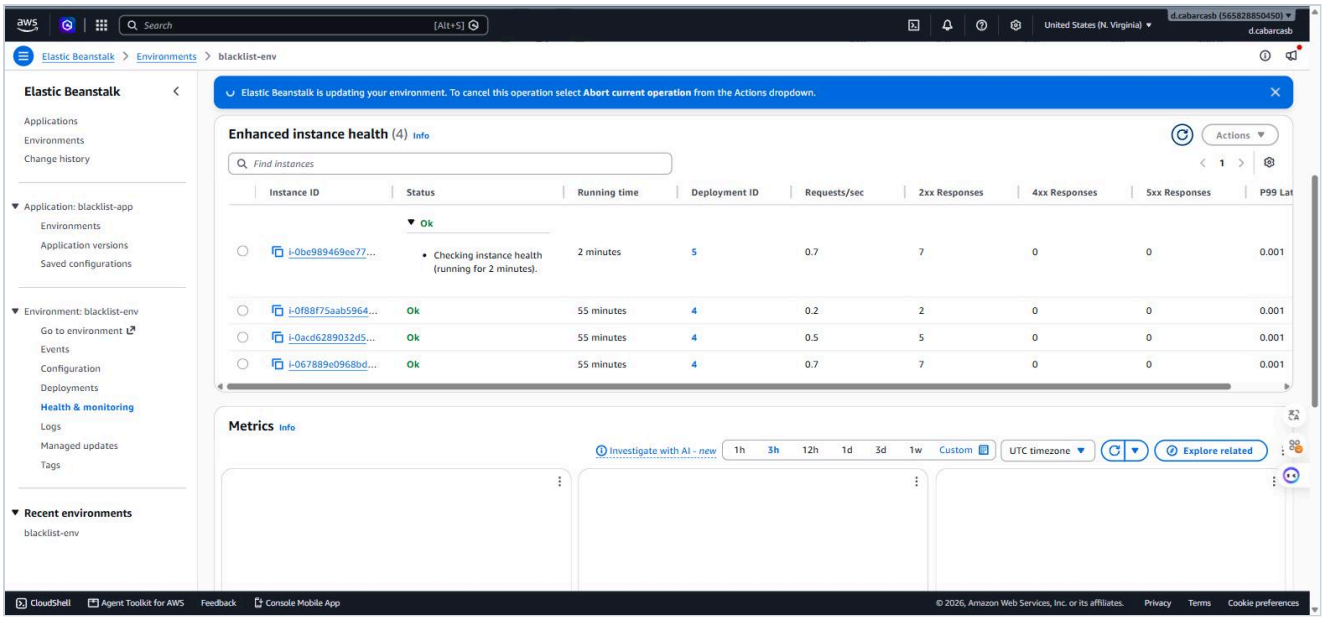


Figura 38. La instancia nueva atendiendo con el Deployment ID 5 junto a las 3 originales.

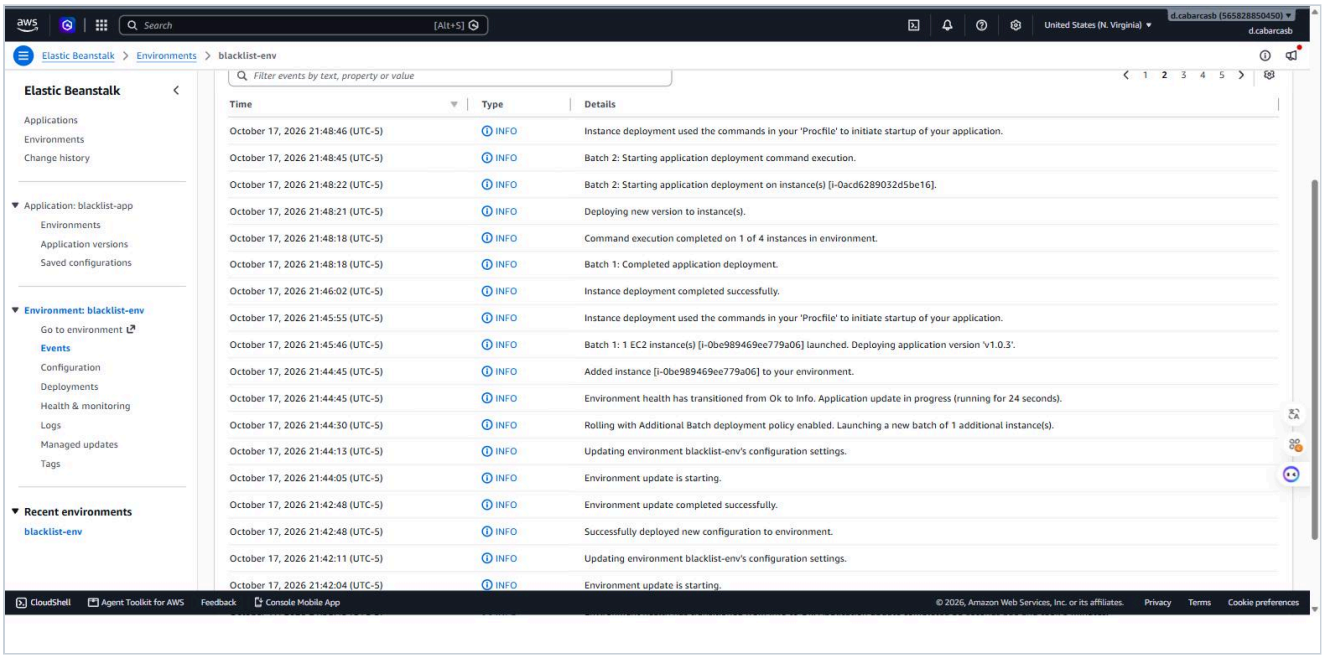


Figura 39. Eventos: lanzamiento del lote adicional e inicio del lote 2.

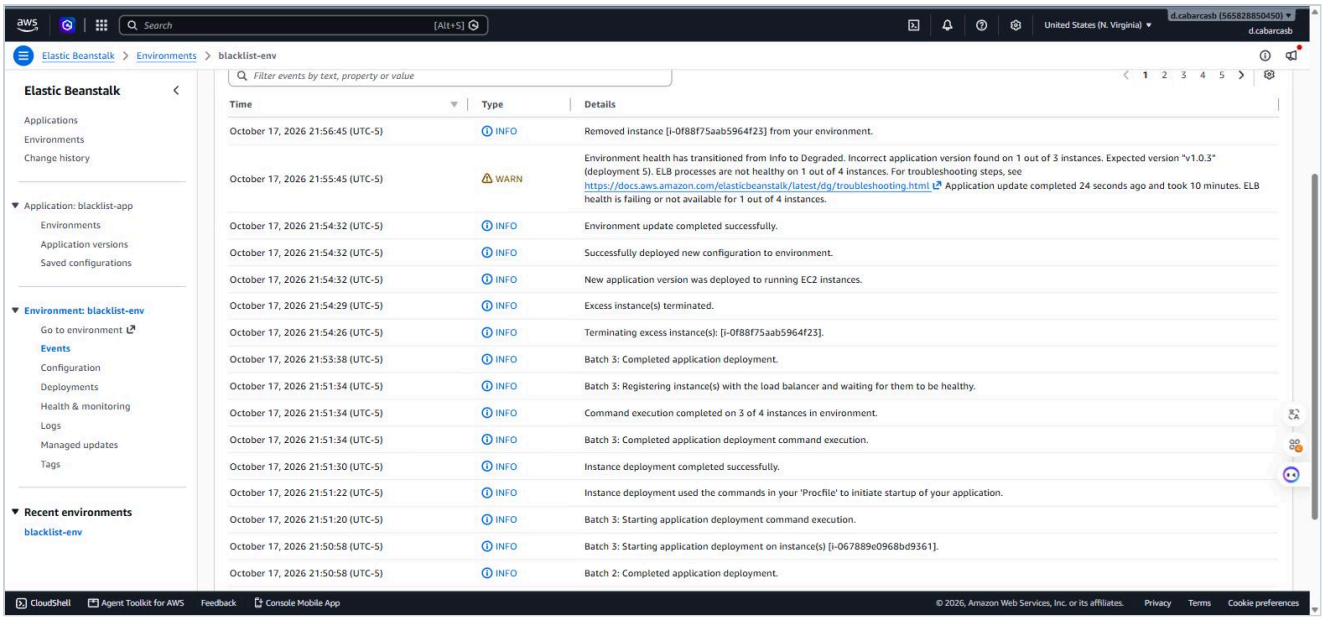


Figura 40. Eventos: terminación de la instancia sobrante y fin del despliegue.

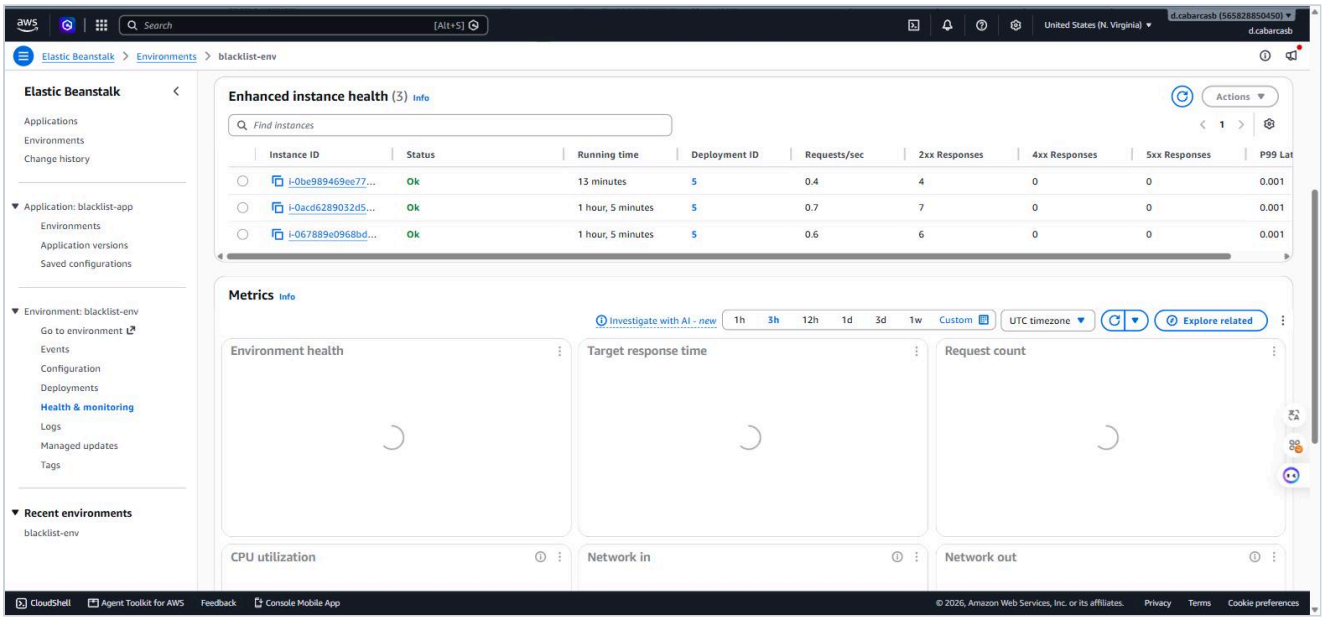


Figura 41. Estado final: 3 instancias (1 nueva y 2 originales) con el Deployment ID 5.

f. Hallazgos

- **Sin caída y sin pérdida de capacidad:** el lote extra compensa la instancia que sale de servicio en cada lote. 0 respuestas 5xx; 3 timeouts aislados (0,4 %).
- Tarda más que Rolling porque suma ~1 min 15 s para lanzar la instancia EC2 nueva antes del primer lote.
- **Costo adicional temporal** de una instancia durante ~10 min.
- Beanstalk no retira la instancia nueva: termina una **original**, la que no se actualizó. El ambiente final mezcla instancias nuevas y viejas.
- Al finalizar, la salud pasó brevemente a *Degraded/Severe* mientras la instancia terminada se desregistraba del ALB (*draining*), sin afectar a los clientes.
- Como en Rolling, conviven dos versiones durante el despliegue.

2.4 Immutable (v1.0.3 → v1.0.4)

a. Instancias utilizadas	3 iniciales + **3 nuevas** en un Auto Scaling group temporal: hasta **6 instancias** en paralelo (3 → 4 → 6 → 3).
b. Validación	Monitor de `/health` (894 peticiones; respondieron **6 hosts distintos**), eventos y EC2 → Instances, donde se ven primero 4 y luego 6 instancias *running*, y al final las 3 originales en *Terminated*. En Health & monitoring las 3 instancias finales tienen el *Deployment ID 6*.
c. Tiempo total	**18 min 06 s** (03:03:28.7 → 03:21:34.7 UTC)
d. Instancias iniciales o nuevas	**Nuevas.** Iniciales `i-0be989469ee779a06`, `i-0acd6289032d5be16` e `i-067889e0968bd9361` (terminadas). Finales `i-0162ce24b7bf06889`, `i-0028d353a4144f877` e `i-0a94de034ecb5cdbd`.

Secuencia observada: (1) se crea un ASG temporal y se lanza **1 instancia de prueba**; (2) cuando pasa los health checks se lanzan las 2 restantes; (3) se espera a que las 3 estén sanas en el ALB, se mueven al ASG permanente y se ejecuta la configuración post-despliegue; (4) se terminan las 3 instancias viejas y el ASG temporal.

e. Capturas

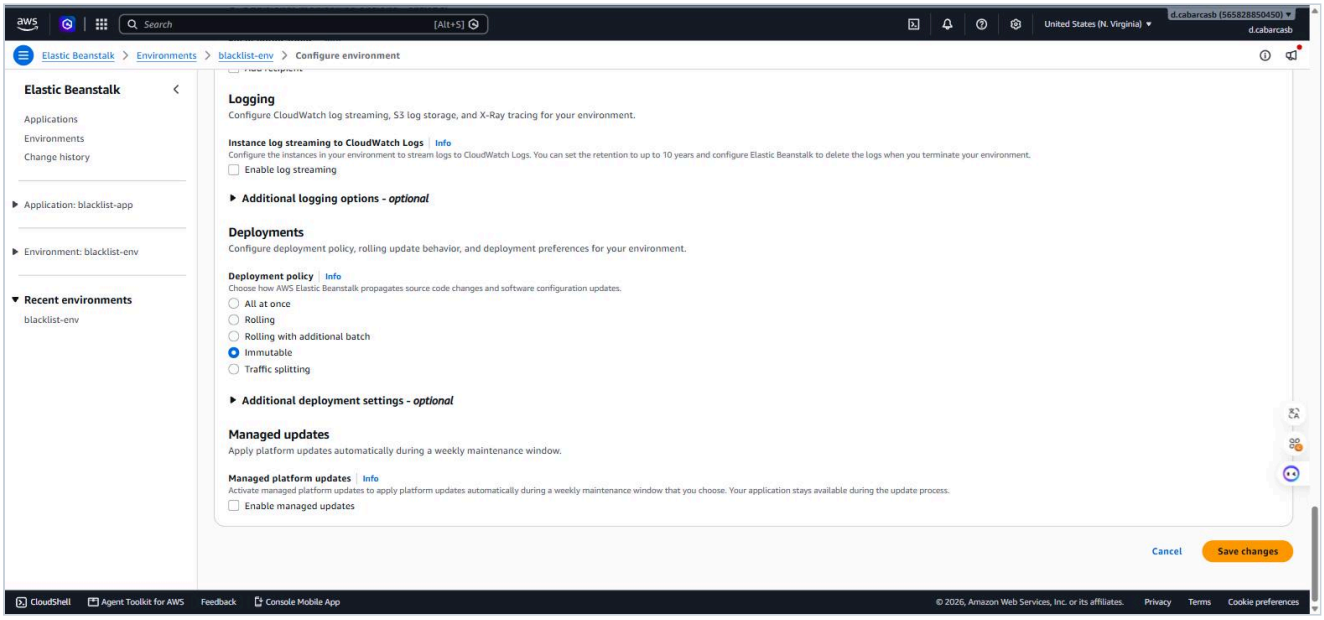


Figura 42. Política de despliegue Immutable.

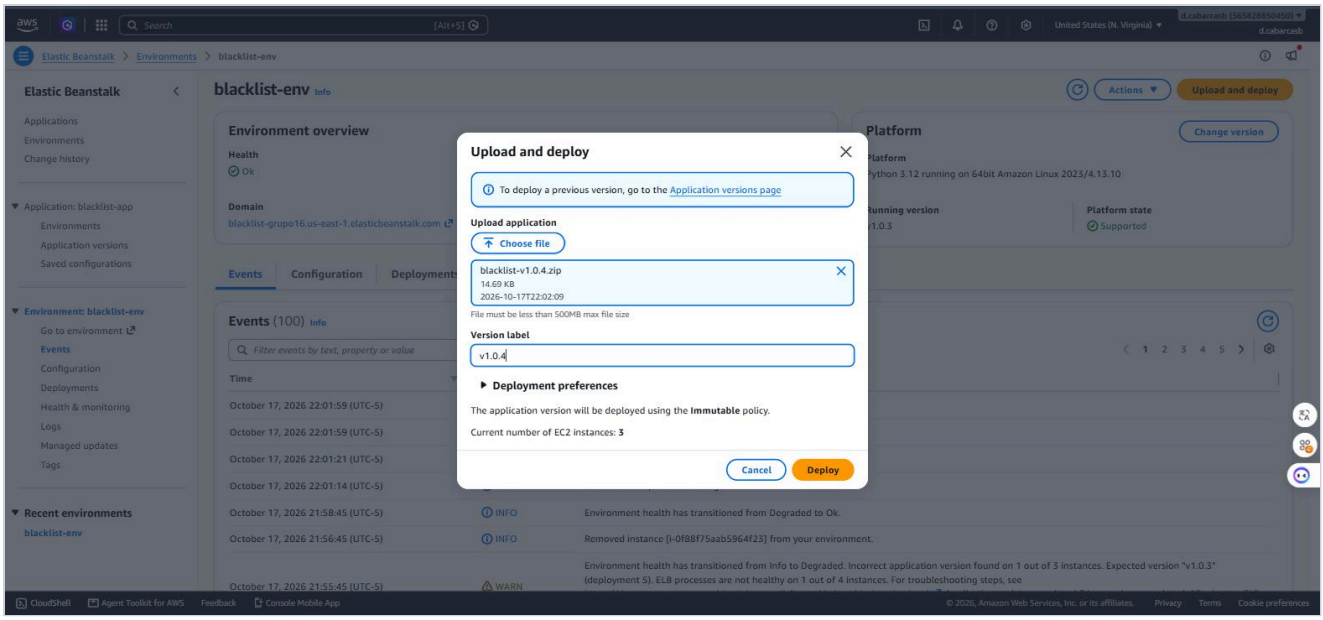


Figura 43. Despliegue de la versión v1.0.4 con la política Immutable.

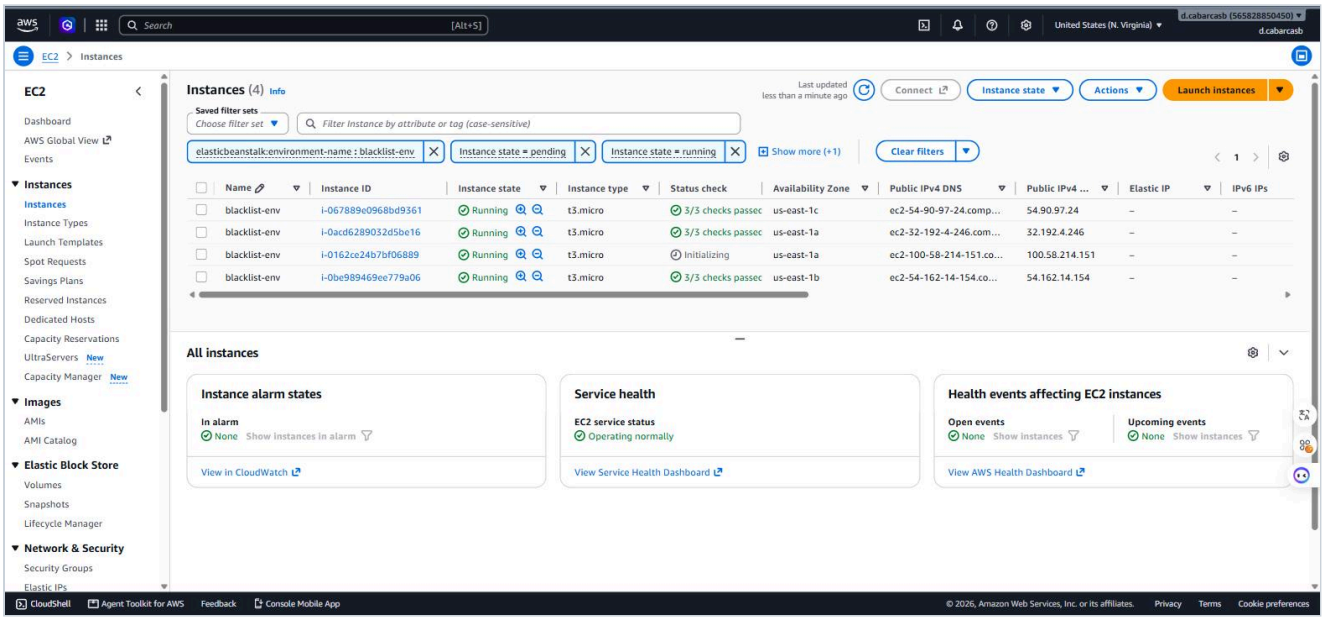


Figura 44. EC2: instancia de verificación (i-0162ce24b7bf06889) junto a las 3 originales.

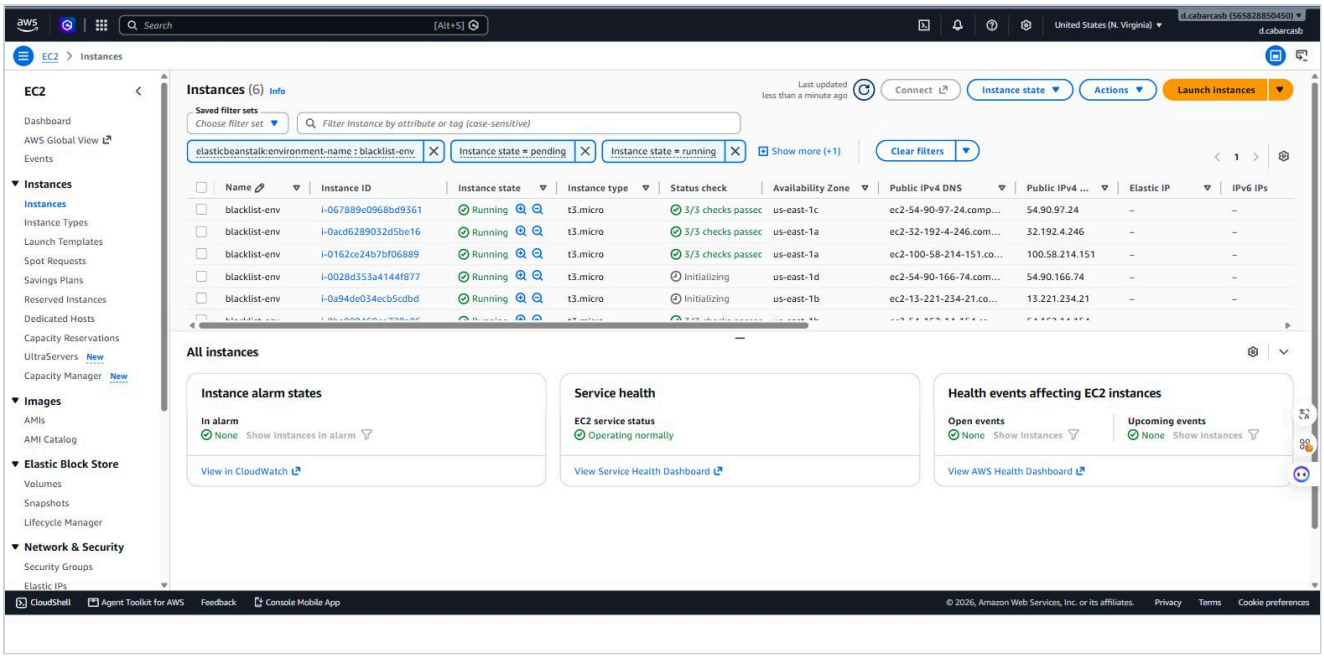


Figura 45. EC2: 6 instancias en ejecución durante el despliegue inmutable.

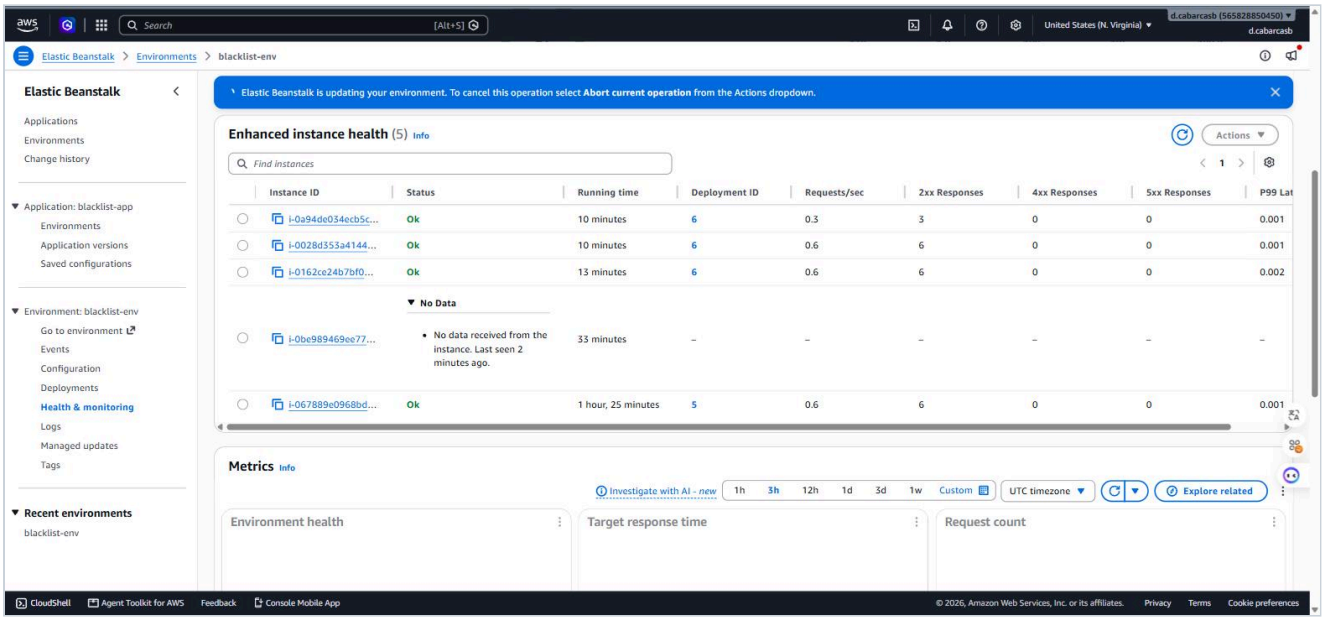


Figura 46. Instancias nuevas con el Deployment ID 6 y las originales saliendo.

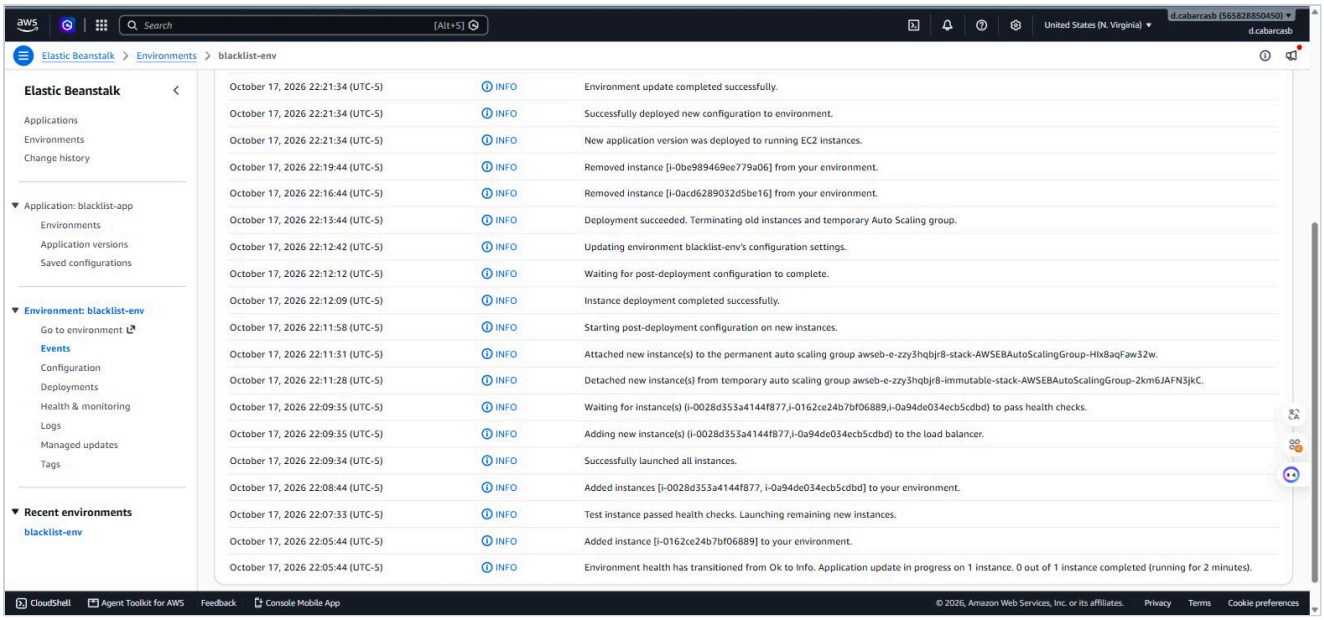


Figura 47. Eventos del despliegue inmutable.

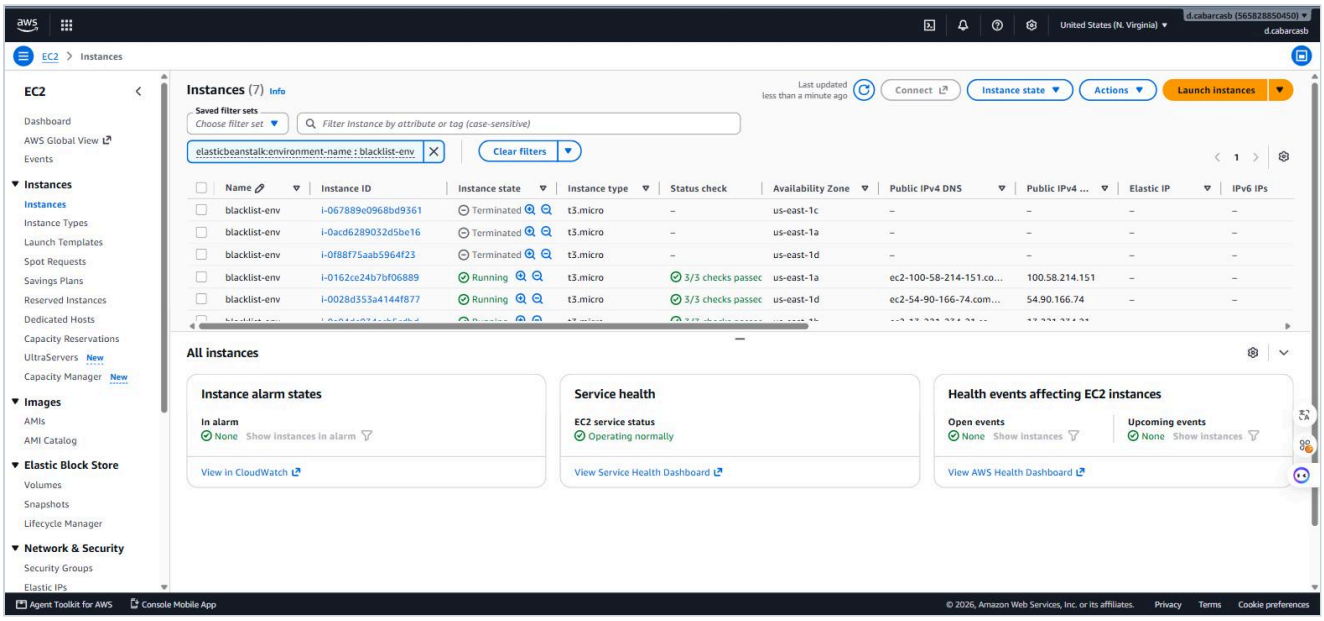


Figura 48. EC2: instancias originales terminadas y nuevas en ejecución.

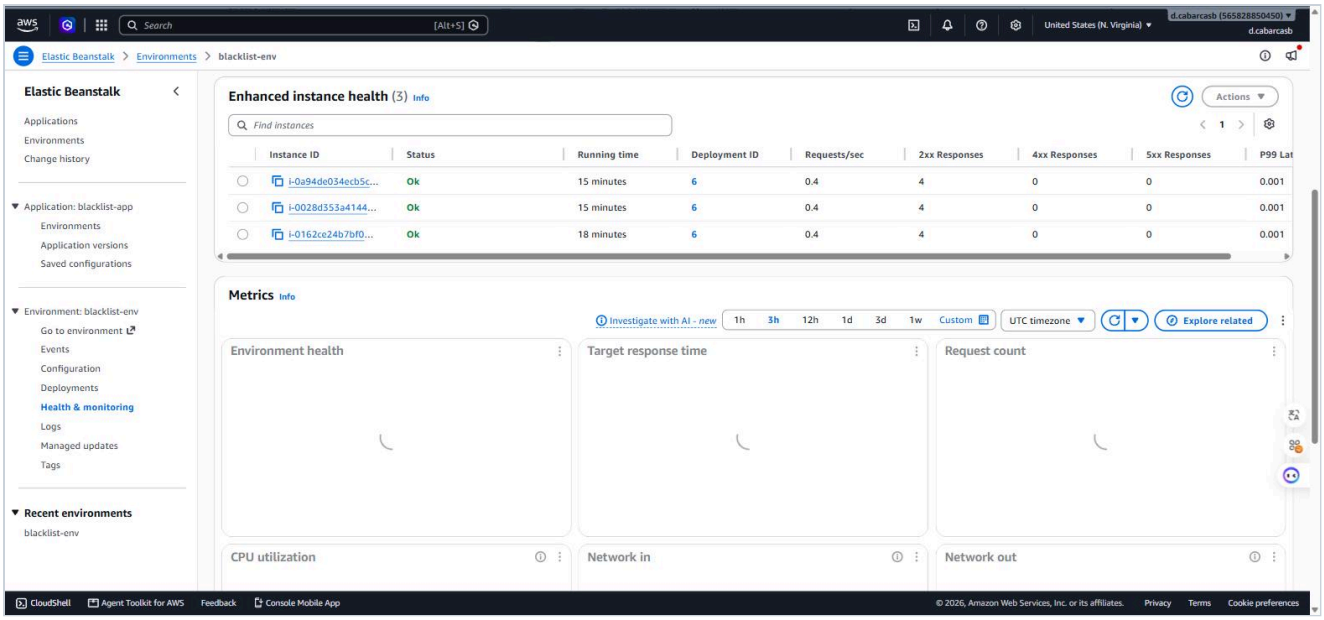


Figura 49. Estado final: 3 instancias nuevas con el Deployment ID 6.

f. Hallazgos

- **Sin caída ni reducción de capacidad:** 0 respuestas 5xx; 3 timeouts aislados (0,3 %). Las instancias viejas atendieron hasta que las nuevas estuvieron sanas.
- Es la estrategia **más lenta** (~18 min): lanzar instancias EC2 nuevas, verificar primero una de prueba y luego terminar las viejas toma tiempo.
- **Mayor costo temporal**: el doble de instancias (6) durante el despliegue; hay que considerar los límites de la cuenta.
- **Rollback más seguro**: si la instancia de prueba falla, solo se descarta el ASG temporal y las instancias originales nunca se tocan.
- Durante la transición conviven brevemente v1.0.3 y v1.0.4, porque las nuevas se registran en el mismo ALB antes de retirar las viejas.
- Las instancias finales quedan recién creadas, lo que evita la deriva de configuración de despliegues in-place anteriores.

3. Comparativo y conclusiones

Criterio	All at once	Rolling	Rolling + lote adicional	Immutable
Tiempo total	**1 min 01 s**	9 min 18 s	10 min 26 s	18 min 06 s
Caída del servicio	Sí (~4 s, HTTP 502)	No	No	No
Capacidad mínima	0 %	66 %	100 %	100 %
Instancias máximas	3	3	4	6
Mezcla de versiones	No	Sí	Sí	Sí (breve)
Instancias finales	Iniciales	Iniciales	Mixtas	Nuevas
Costo extra	Ninguno	Ninguno	1 instancia ~10 min	3 instancias ~15 min
Rollback	Redesplegar todo	Redesplegar	Redesplegar	Descartar el ASG temporal

- **All at once** conviene en ambientes de desarrollo o cuando se acepta una breve indisponibilidad.
- **Rolling** evita la caída sin costo extra, a cambio de capacidad reducida y versiones mezcladas.
- **Rolling with additional batch** mantiene la capacidad completa con un costo extra pequeño; es útil en producción con tráfico constante.
- **Immutable** es la más segura (no toca las instancias en servicio y el rollback es inmediato), a cambio de ser la más lenta y costosa.

> Los logs completos del monitor de `/health` de cada estrategia están en `entregas/monitoreo/` del repositorio, y el detalle de cada estrategia en `entregas/estrategias-despliegue.md`.